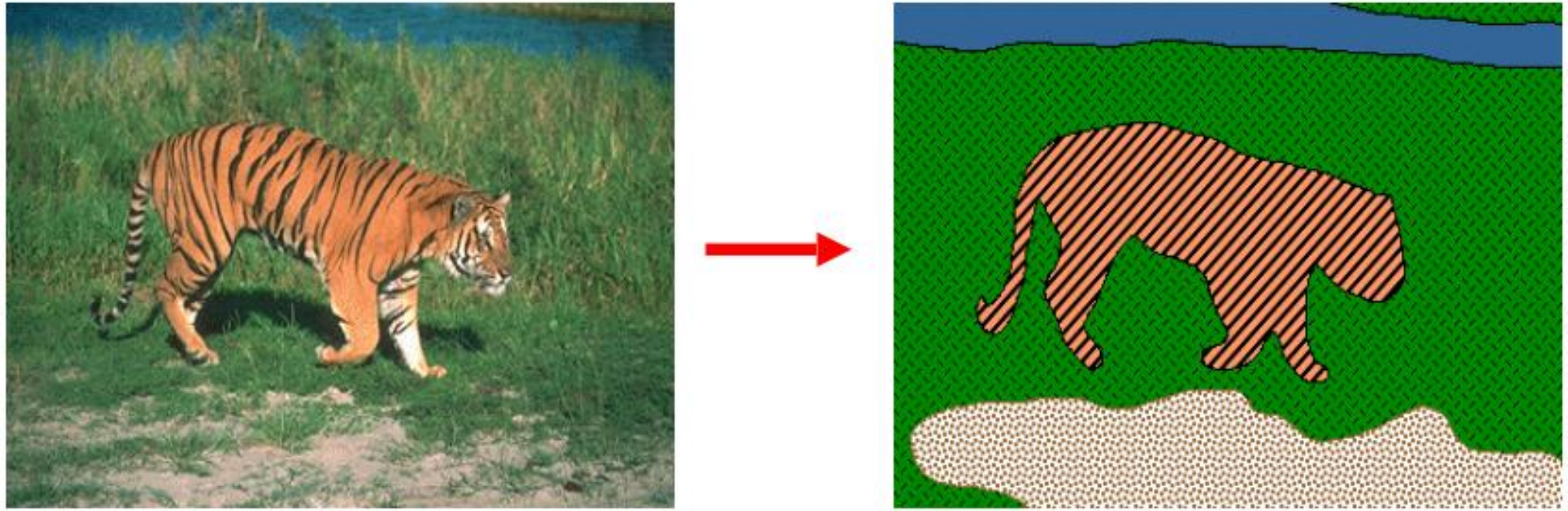
Computer Vision

Saint Joseph University of Beirut
Ecole Supérieure d'Ingénieurs de Beyrouth
CCE Program

Université Saint-Joseph de Beyrouth
Faculté d'ingénierie et d'architecture
École supérieure d'ingénieurs de Beyrouth

- **Introduction to Computer Vision and Image Formation**
- **Image Filtering and Edge Detection**
- **Feature Detection and Matching**
- **Object Detection with Traditional Methods**
- **Image Segmentation Techniques**
- Introduction to Deep Learning for Vision
- CNN Architectures and Image Classification
- Vision Transformers: Theory and Applications in Vision
- Pretrained Models and Annotation Tools
- Object Detection with Deep Learning
- Dimensionality Reduction using Autoencoders and CAEs
- 3D Vision and Depth Estimation
- Object Tracking

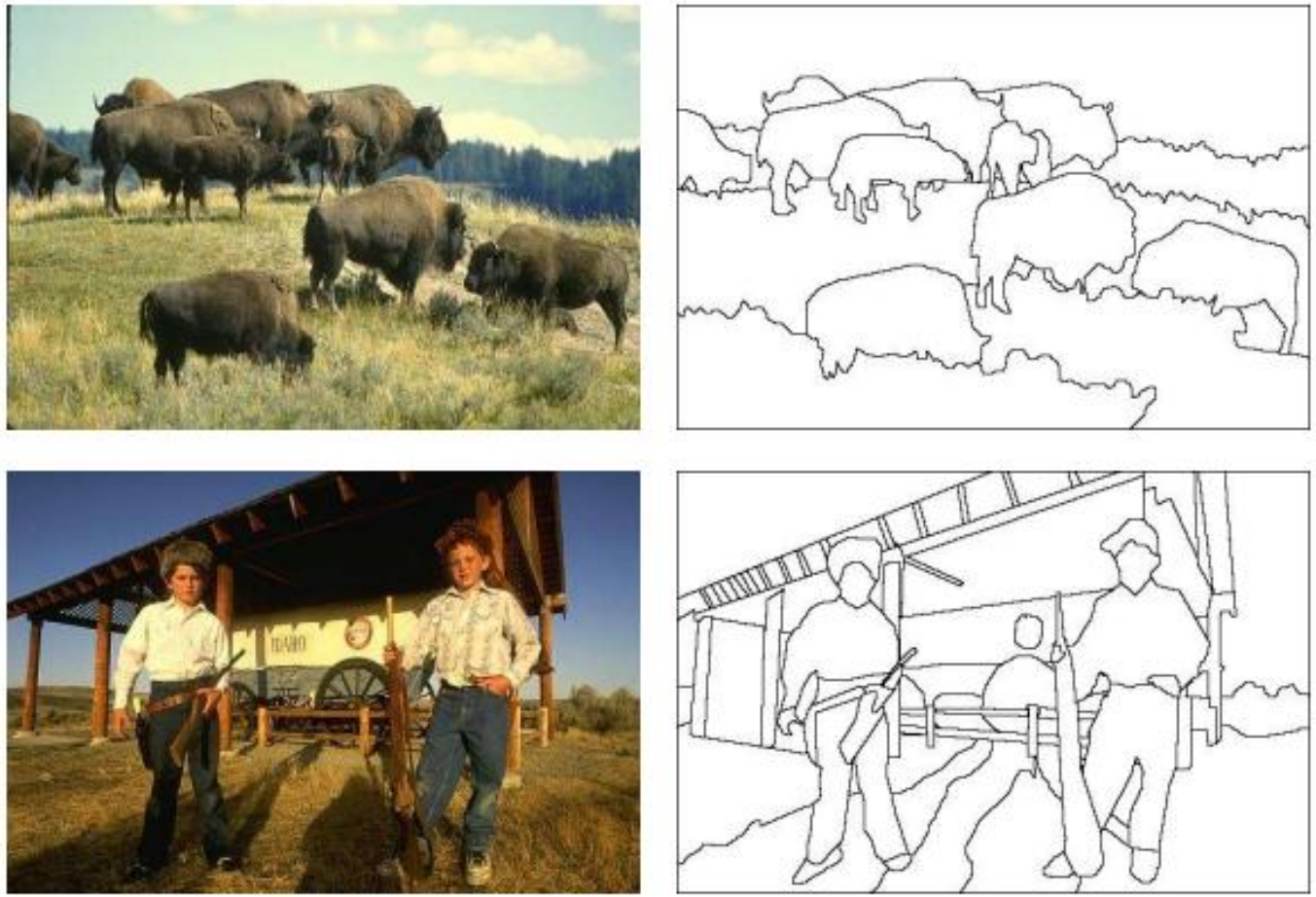
Definition



"Segmentation is the process of partitioning an image into semantically interpretable regions."

H. Barrow and J. Tennenbaum, 1978

Definition

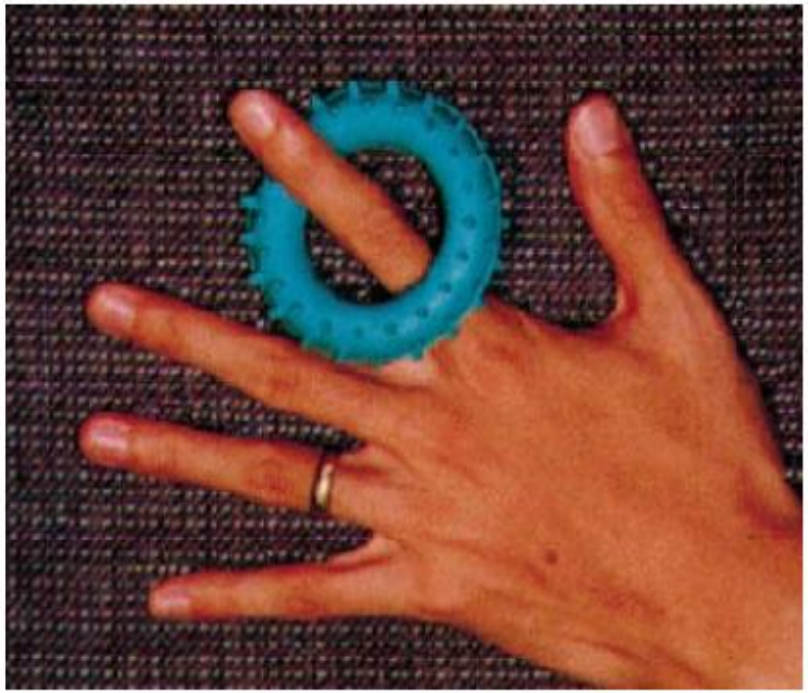


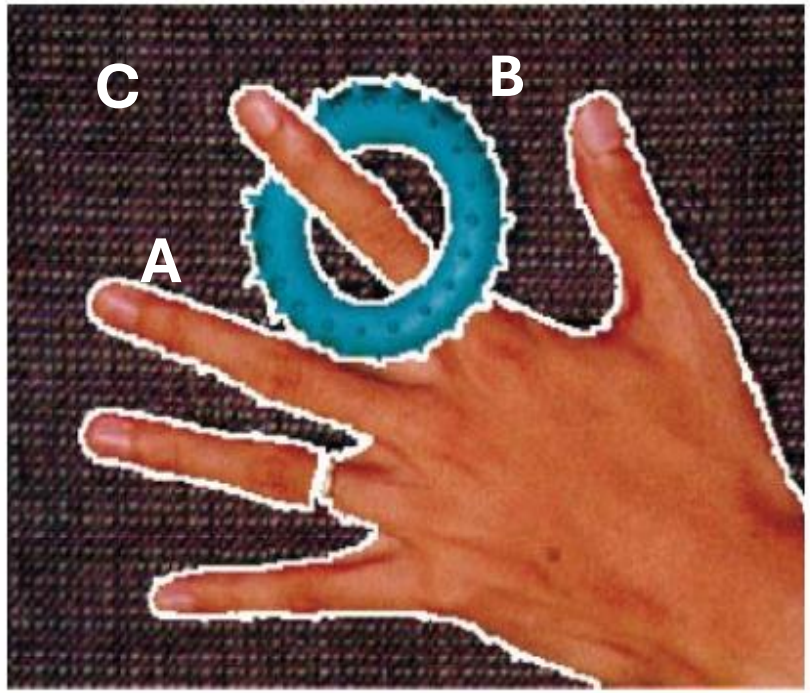
Separate image into multiple, coherent segments (sets of pixels “image objects”!)

Definition

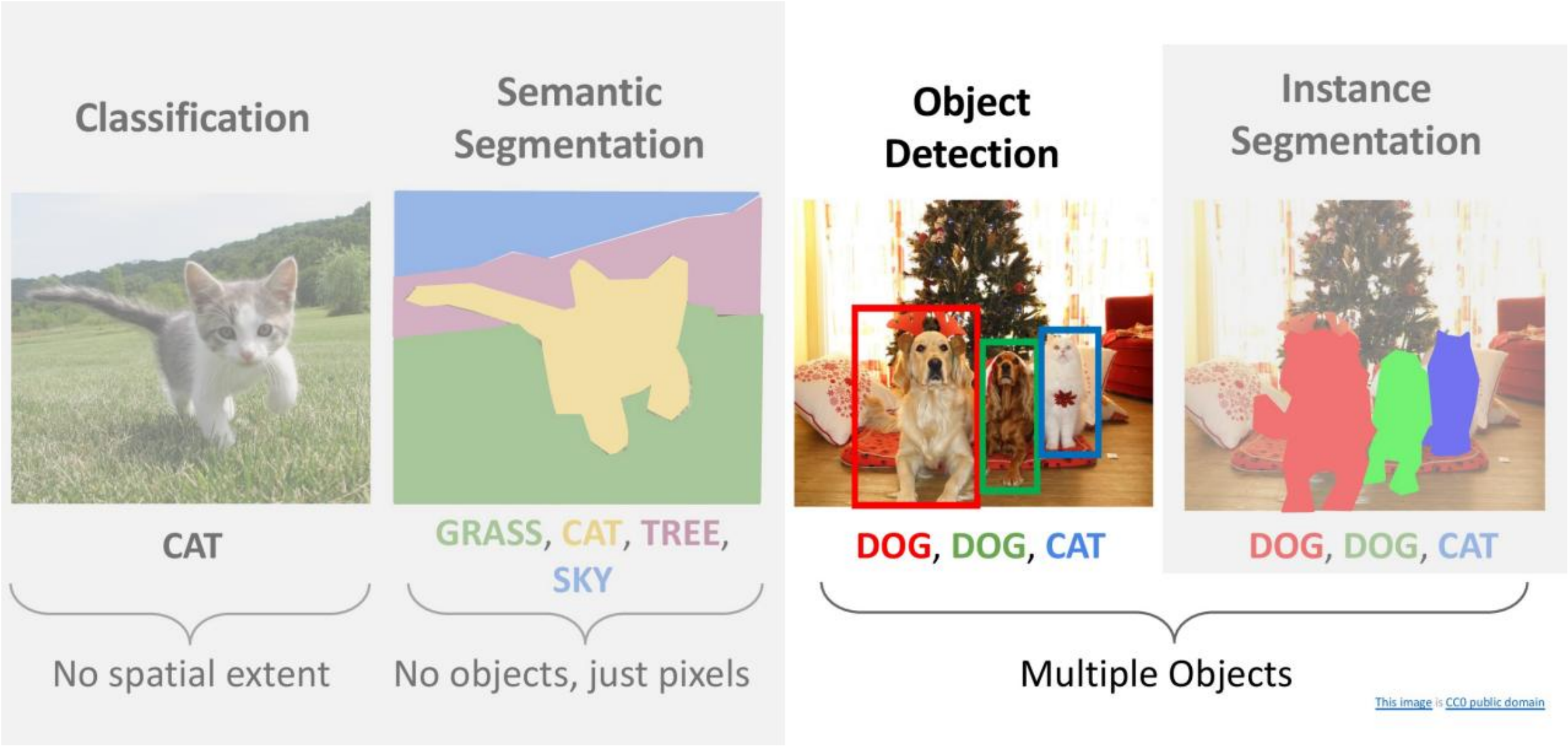
Goal: Separate / group image pixels to objects

What is an object?





Types of Segmentation



Goals of Segmentation

- Obtain primitives for other tasks

  Simplify and/or change the representation of an image into something that is more meaningful and easier to analyze

- Perceptual organization, recognition

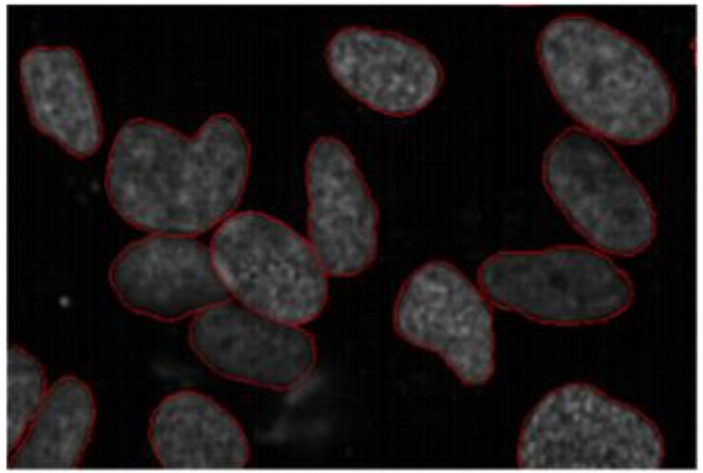
  Used to locate objects and boundaries (lines, curves, etc.).

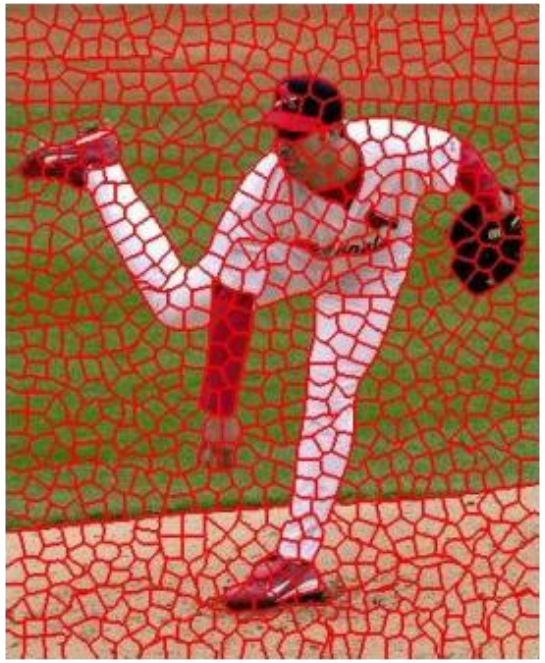
- Graphics, image manipulation, 3D reconstruction.

Goals of Segmentation

Goal 1: Primitives for other tasks

- Group together similar-looking pixels for efficiency of further processing
- Simplify and/or change the representation of an image into something that is more meaningful and easier to analyze
- Used to locate objects and boundaries (lines, curves, etc.)

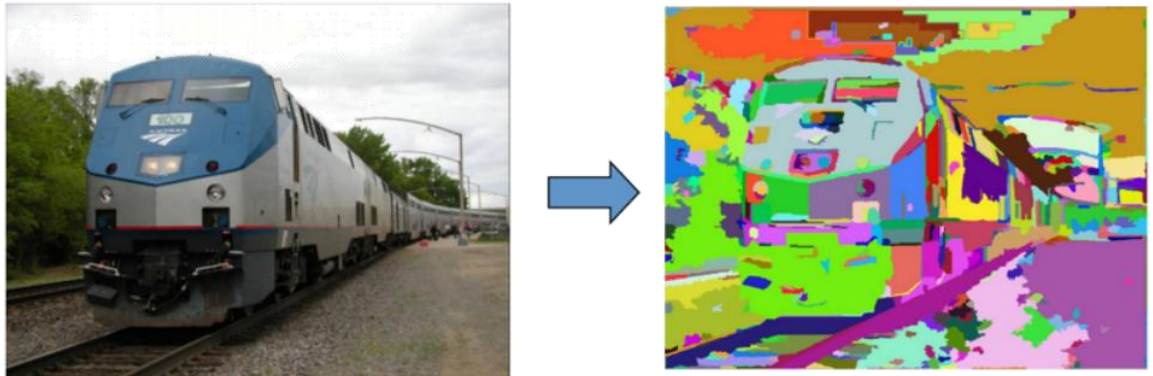




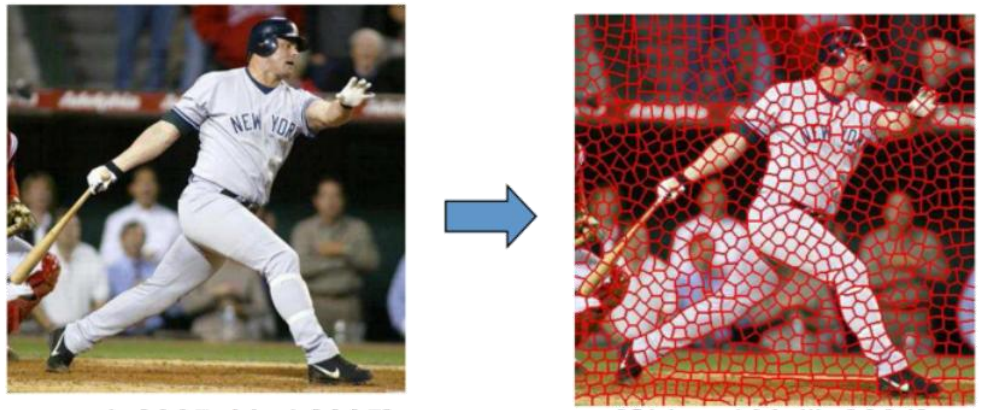
Application

Better Classification

Speed up recognization



[Felzenszwalb and Huttenlocher 2004]



[Hoiem et al. 2005, Mori 2005]

[Shi and Malik 2001]

Slide: Derek Hoiem

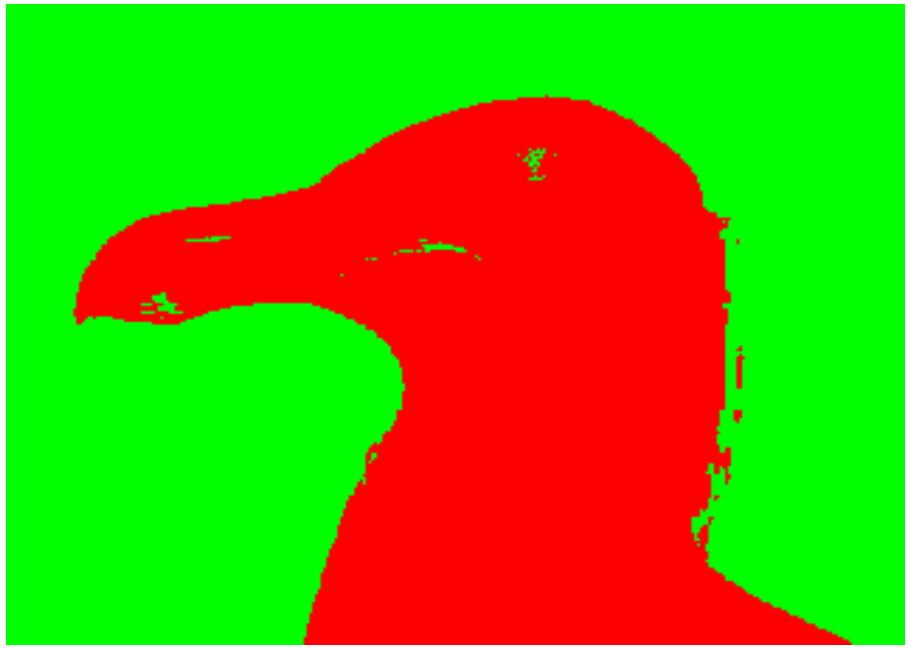
Segmentation

Can you think of the simplest segmentation method?

Segmentation

Can you think of the simplest segmentation method?

Just threshold the intensity (color).



Threshold on H-S (from HSV)

Approaches to segmentation

- Region-based segmentation
  - simple way to segment different objects could be to use their pixel values → use threshold values

- Edge Detection Segmentation
  - We can make use of the discontinuity between pixels to detect edges and hence define a boundary of the object for segmentation ?

- Segmentation as clustering.
  - dividing the population (data points) into a number of groups,
  - data points in the same groups → more similar

- Segmentation as graph partitioning.

Segmentation as Thresholding

Segmentation as Thresholding

Thresholding is one of the easiest methods of image segmentation where a threshold is set for dividing pixels into two classes. Pixels that have values greater than the threshold value are set to 1 while pixels with values lesser than the threshold value are set to 0.

The image is thus converted into a binary map, resulting in the process often termed binarization. Image thresholding is very useful in case the difference in pixel values between the two target classes is very high, and it is easy to choose an average value as the threshold.

Thresholding is often used for image binarization so that further algorithms like contour detection and identification that work only on binary images can be used

Segmentation as Thresholding

- **Grayscale image ⇒ Binary mask**
- **Different variants**
  - **One-sided**

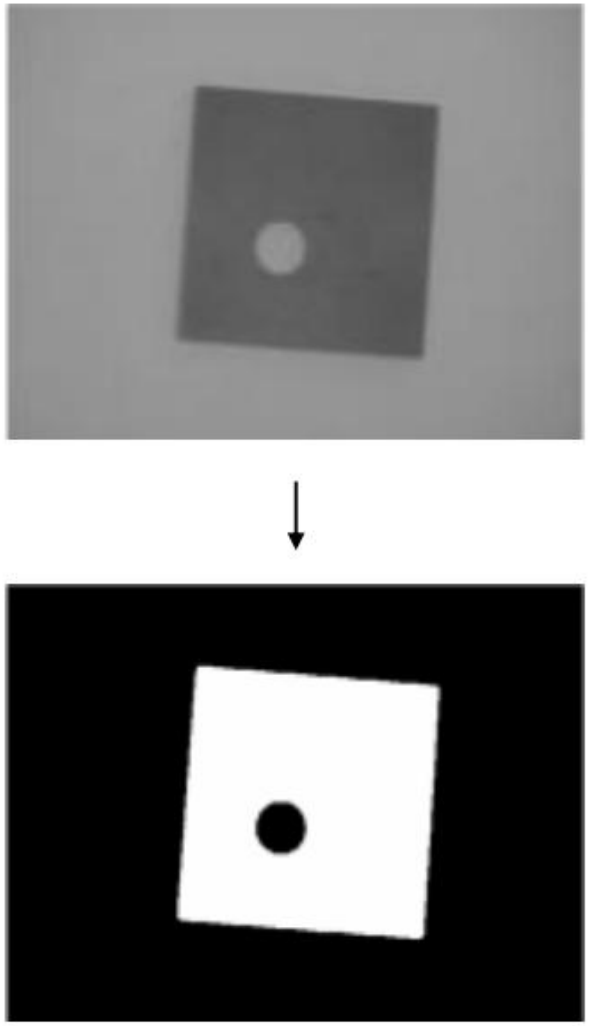
$$F_T[i,j] = \begin{cases} 1, & \text{if } F[i,j] \geq T \\ 0, & \text{otherwise} \end{cases}$$

  - **Two-sided**

$$F_T[i,j] = \begin{cases} 1, & \text{if } T_1 \leq F[i,j] \leq T_2 \\ 0, & \text{otherwise} \end{cases}$$

  - **Set membership**

$$F_T[i,j] = \begin{cases} 1, & \text{if } F[i,j] \in Z \\ 0, & \text{otherwise} \end{cases}$$



Segmentation as Thresholding / Otsu's method

- Once you understand global thresholding, the next natural question is: *"Can the computer choose the best threshold value for me?"*
- That is exactly what Otsu's method tries to do. Instead of manually picking a number like 120 or 160
- Otsu looks at the entire image histogram and finds a threshold that best separates the pixels into two groups: background and foreground.
- It aims to find the threshold value that maximizes the between-class variance, essentially creating the best possible separation between the two classes (foreground and background).

Segmentation as Thresholding / Otsu's method

Step 0 — Compute the Histogram

Image intensities range from **0 → L-1**
(Usually L = 256)
Histogram: h(i)

Step 1 — Normalize Histogram
Convert histogram into probability distribution.

$$P(i) = \frac{h(i)}{N}$$

Meaning: Probability that a random pixel has intensity i.

Segmentation as Thresholding / Otsu's method

Step 2 — Try Each Possible Threshold T

We test every threshold:

T = 0 → L-1

For each threshold, we split pixels into two classes.

Class 1 (Background)

Pixels with intensity:

$$0 \leq i \leq T$$

Class 2 (Foreground)

Pixels with intensity:

$$T + 1 \leq i \leq L - 1$$

Segmentation as Thresholding / Otsu's method

Step 3 — Compute Class Probabilities

These represent how many pixels belong to each class.

Probability of Class 1

$$w_1(T) = \sum_{i=0}^{T} P\,(i)$$

Probability of Class 2

$$w_2(T) = \sum_{i=T+1}^{L-1} P\,(i)$$

Note:

$$w_1 + w_2 = 1$$

Segmentation as Thresholding / Otsu's method

Step 4 — Compute Class Means

Mean Intensity of Class 1

$$\mu_1(T) = \frac{\sum_{i=0}^{T} i\, P(i)}{w_1(T)}$$

Mean Intensity of Class 2

$$\mu_2(T) = \frac{\sum_{i=T+1}^{L-1} i\, P(i)}{w_2(T)}$$

Segmentation as Thresholding / Otsu's method

Step 5 — Compute Between-Class Variance

Otsu measures how separated the two classes are using:

$$\sigma_b^2(T) = w_1(T)\, w_2(T)\, \left[\mu_1(T) - \mu_2(T)\right]^2$$

Interpretation

Large value means:

✓ Class sizes are significant

✓ Class means are far apart

Which means strong separation.

Segmentation as Thresholding / Otsu's method

Step 6 — Find Optimal Threshold

Search for threshold that maximizes between-class variance:

$$T^* = \arg\max_T \quad \sigma_b^2(T)$$

🧩 Step 7 — Apply Threshold

Final segmentation:

If pixel intensity ≥ T* → foreground

Else → background

Segmentation as Thresholding / Otsu's method

1. Compute histogram h(i)
2. Compute probability P(i)

3. For T = 0 to L-1:
   Compute w1(T)
   Compute w2(T)
   Compute μ1(T)
   Compute μ2(T)
   Compute $\sigma_b^2(T)$

4. Choose T giving maximum σ_b^2
5. Threshold image using T

Source image

Image after thresholding

Segmentation as clustering

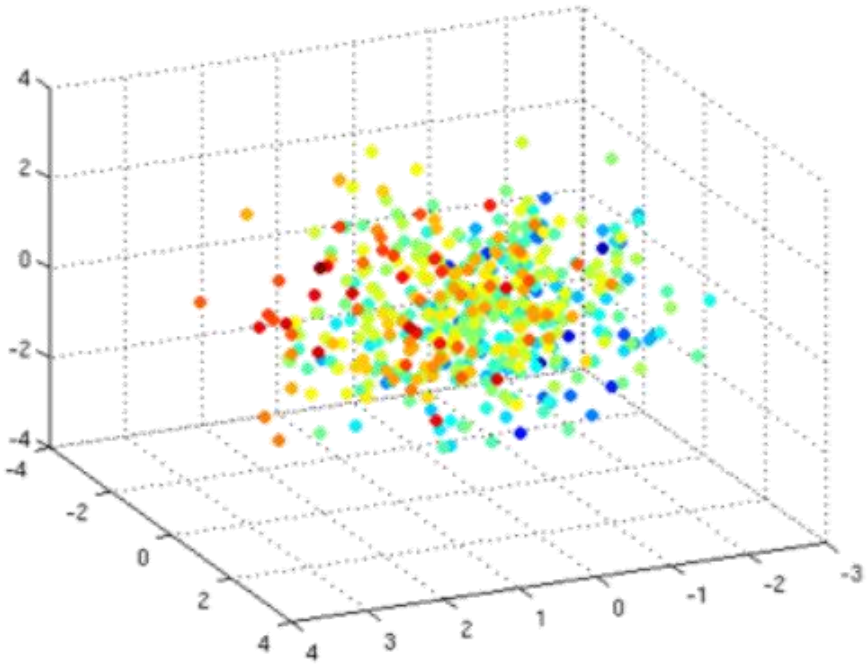
1. K-Means
2. Mean Shift

Segmentation as clustering

Pixels are points in a high-dimensional space

- color: 3d
- color + location: 5d

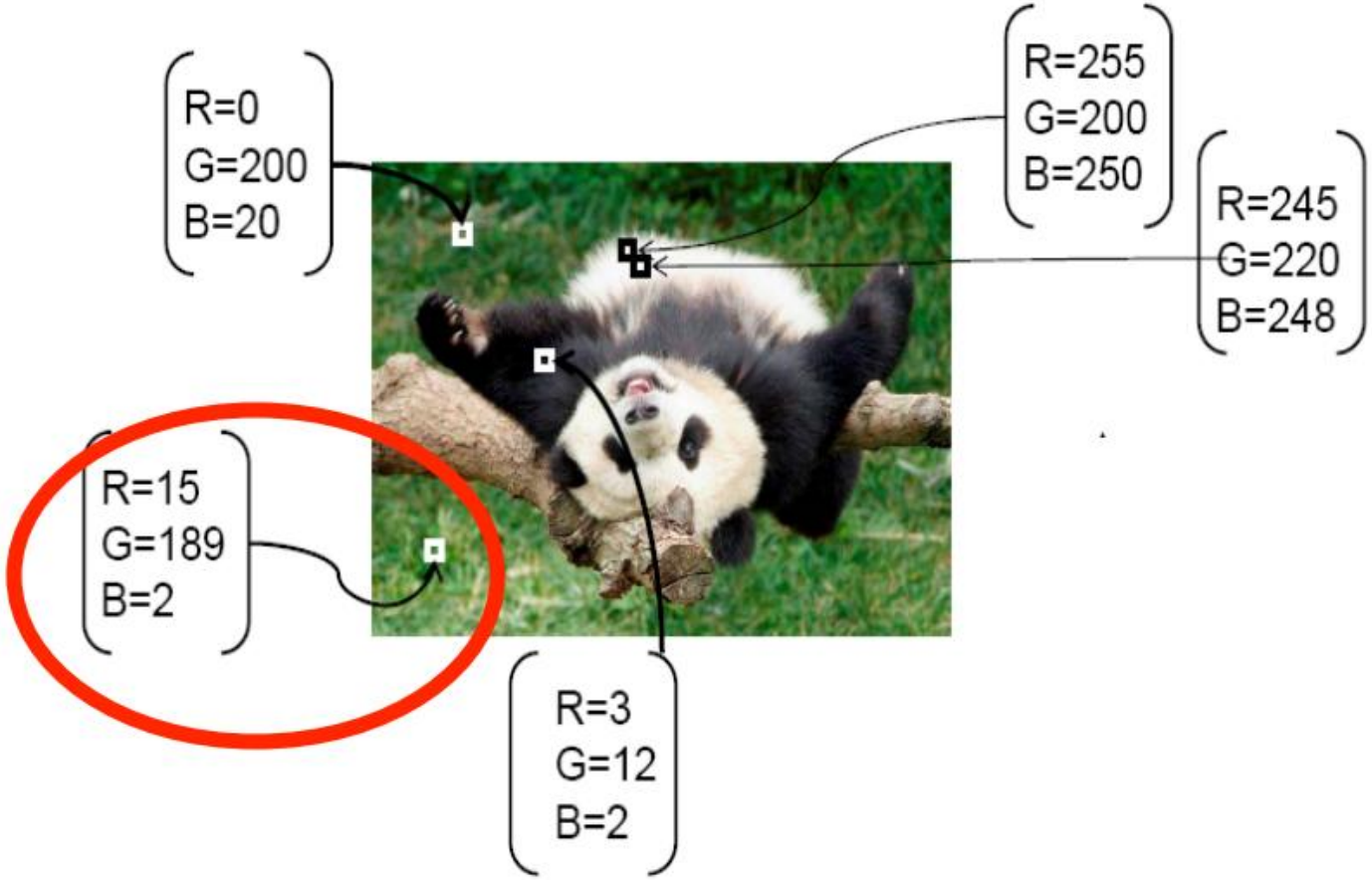
Cluster pixels into segments



Segmentation as clustering/Feature Space

Every token is identified by a set of salient visual characteristics. For example:

- Position
- Color
- Texture
- Motion vector
- Size, orientation
  (if token is larger than a pixel)



Segmentation as clustering/Feature Space

What Segmentation Means Here

Group pixels that are similar and give them the same label.

Example:
All sky pixels → one group
All road pixels → another group
All grass pixels → another group

K-means helps us do exactly that.

Segmentation as clustering/Feature Space

The Main Idea of K-Means

K-means says:
"Let me divide all pixels into **K groups** based on similarity."
Where:
K = number of segments you want.

For example:
K = 2 → separate foreground and background
K = 3 → maybe sky, ground, object
K = 5 → more detailed segmentation

Segmentation as clustering/Feature Space

How The Algorithm Works (Intuition First)

Imagine all pixels are dots floating in color space.

Choose K random centers

If K = 3, the algorithm randomly chooses 3 colors as group centers.

Example:

Center 1 → reddish color
Center 2 → bluish color
Center 3 → greenish color

Segmentation as clustering/Feature Space

Assign Each Pixel To The Closest Center

For every pixel:

Measure distance between pixel color and each center.

Usually we use Euclidean distance:

$$distance = \sqrt{\left(R_1 - R_2\right)^2 + \left(G_1 - G_2\right)^2 + \left(B_1 - B_2\right)^2}$$

Then:

Pixel joins the closest center group.

Segmentation as clustering/Feature Space

Update Centers

Now each group contains many pixels.

We recompute the center as: The average color of all pixels in that group.

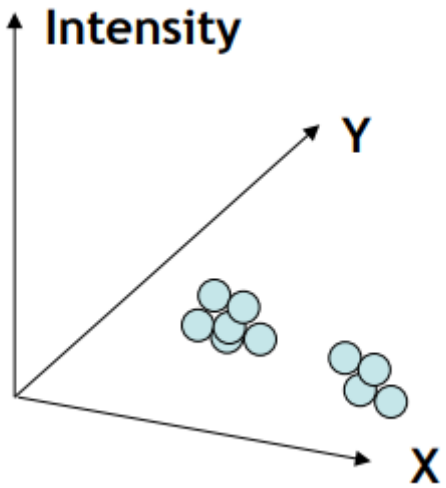
Repeat

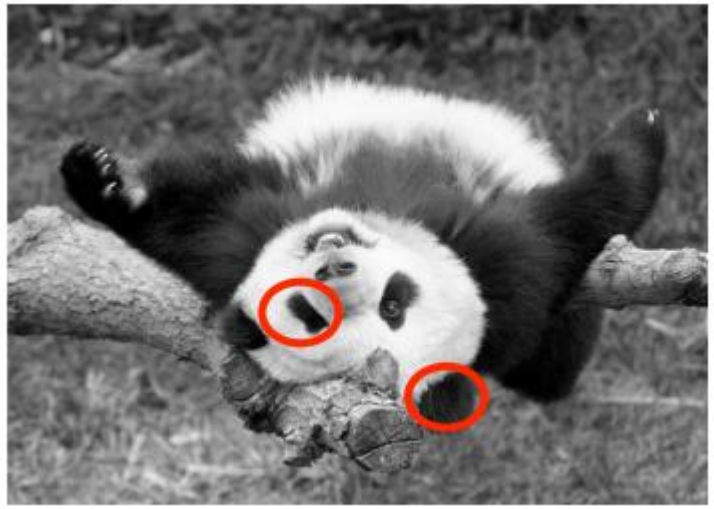
The algorithm keeps repeating:

- Assign pixels to nearest center
- Update centers
- Until groups stop changing.

Segmentation as clustering/Feature Space

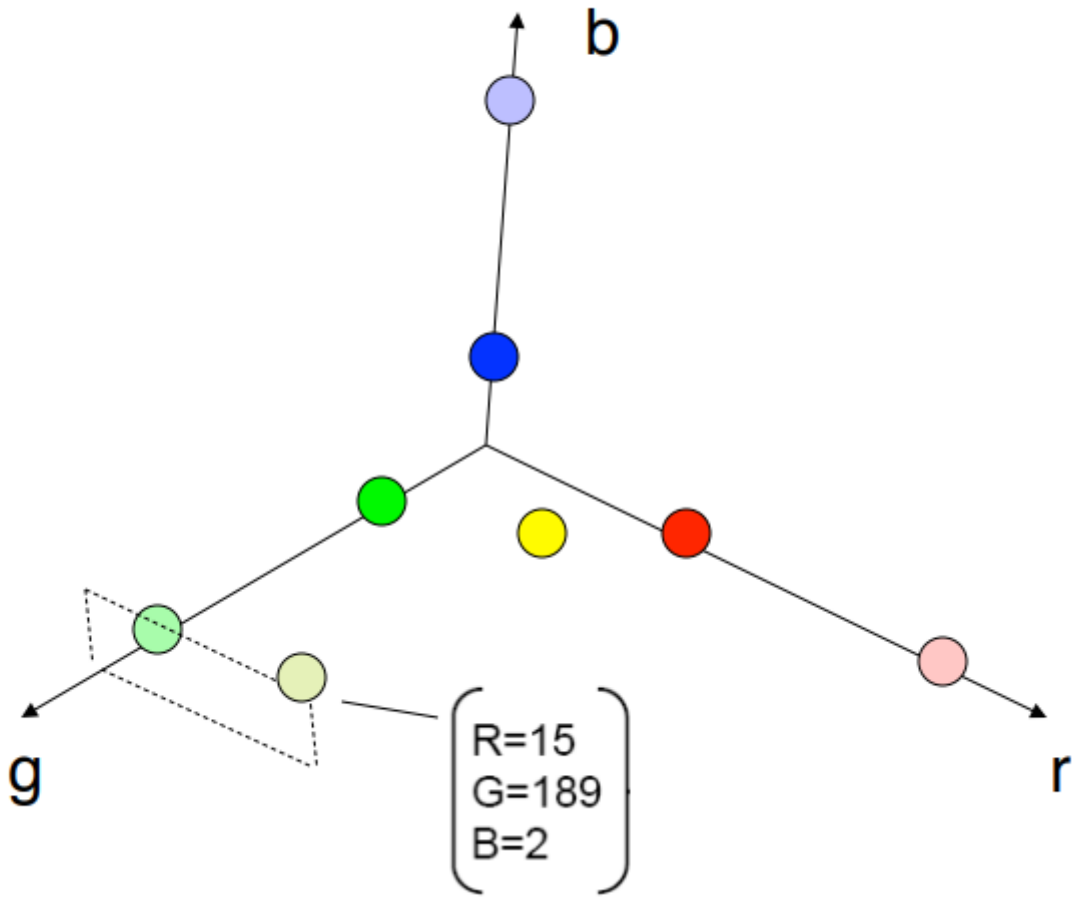
- Depending on what we choose as the feature space, we can group pixels in different ways.
- Grouping pixels based on intensity + position similarity





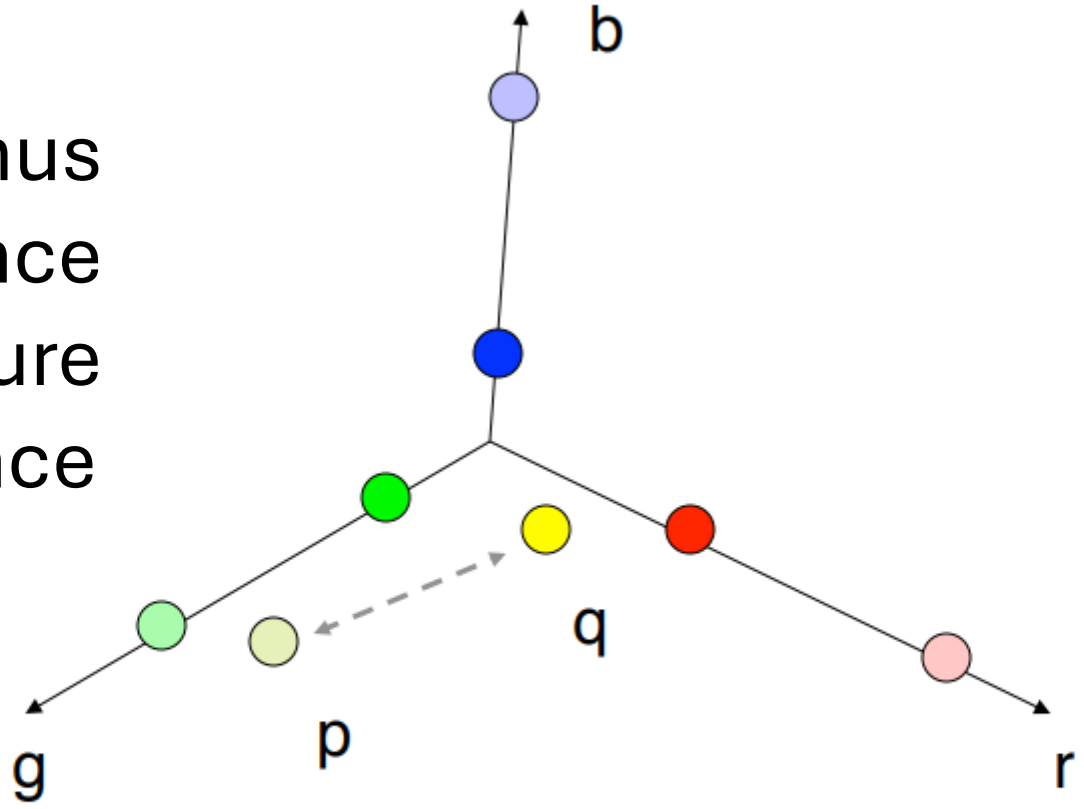
⇒ Way to encode both similarity and proximity.

Segmentation as clustering/Feature Space



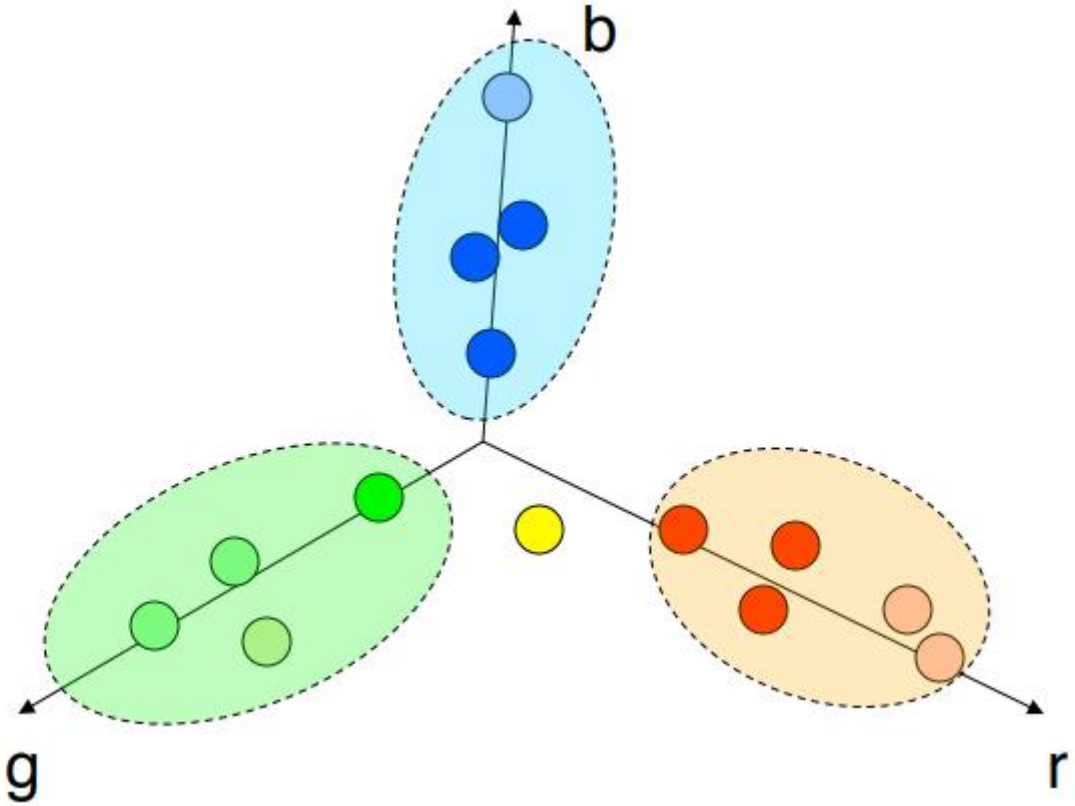
Segmentation as clustering/Feature Space

Token similarity is thus measured by distance between points ("feature vectors") in feature space



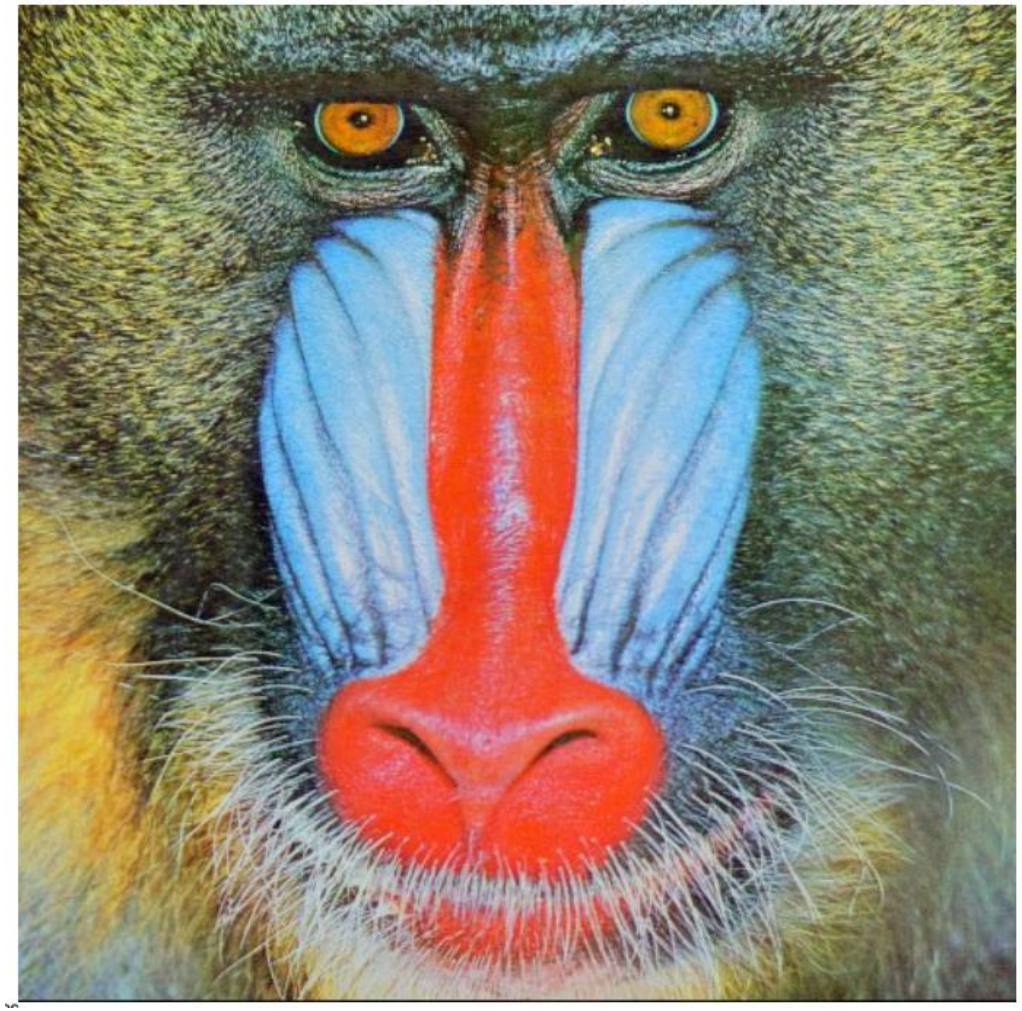
$$\sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \cdots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}.$$

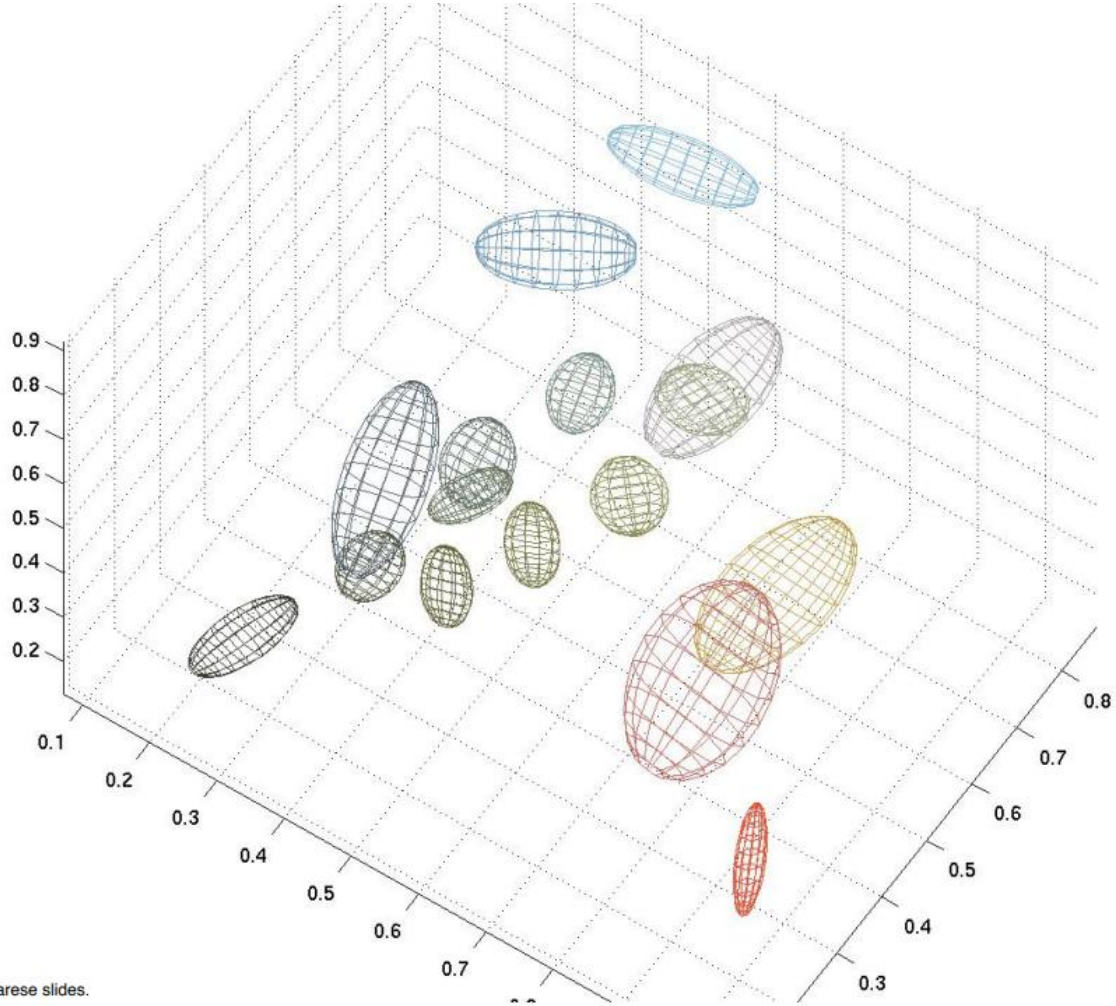
Segmentation as clustering / Feature Space



Cluster together tokens with high similarity

Segmentation as clustering / Feature Space





Savarese slides.

Segmentation as clustering / K-Means

Algorithm:

1. Randomly initialize the cluster centers, $c_1 \dots c_k$
2. Given cluster centers, determine points in each cluster
   - For each point p, find the closest c_i . Put p into cluster i
3. Given points in each cluster, solve for c_i
   - Set c_i to be the mean of points in cluster i
4. If c_i have changed, repeat Step 2

$$\sum_{i=1}^{k} \sum_{x \in C_i} \|x - c_i\|^2$$

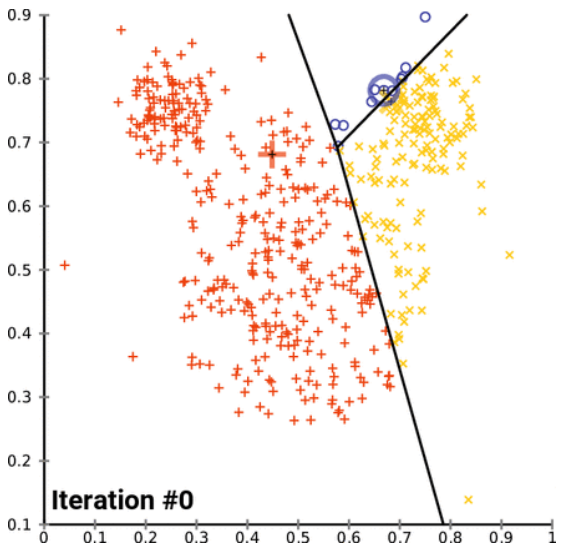
Properties:

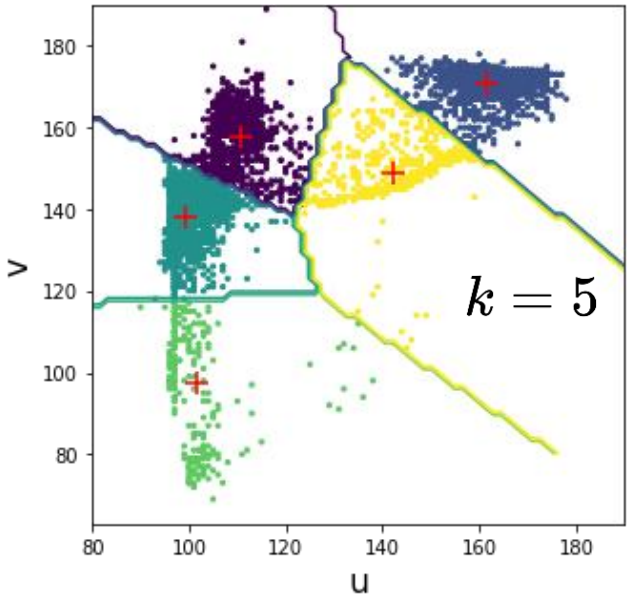
- Will always converge to some solution
- Can be a "local minimum"
- Does not always find the global minimum of objective function

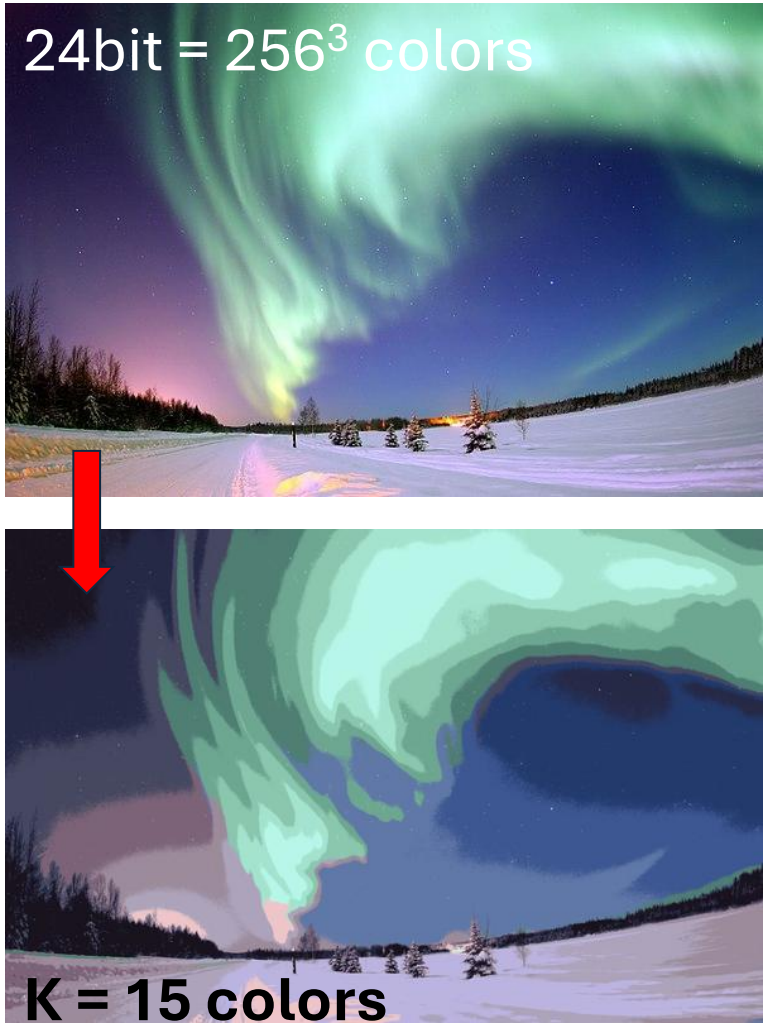
Segmentation as clustering / K-Means

k-Means:

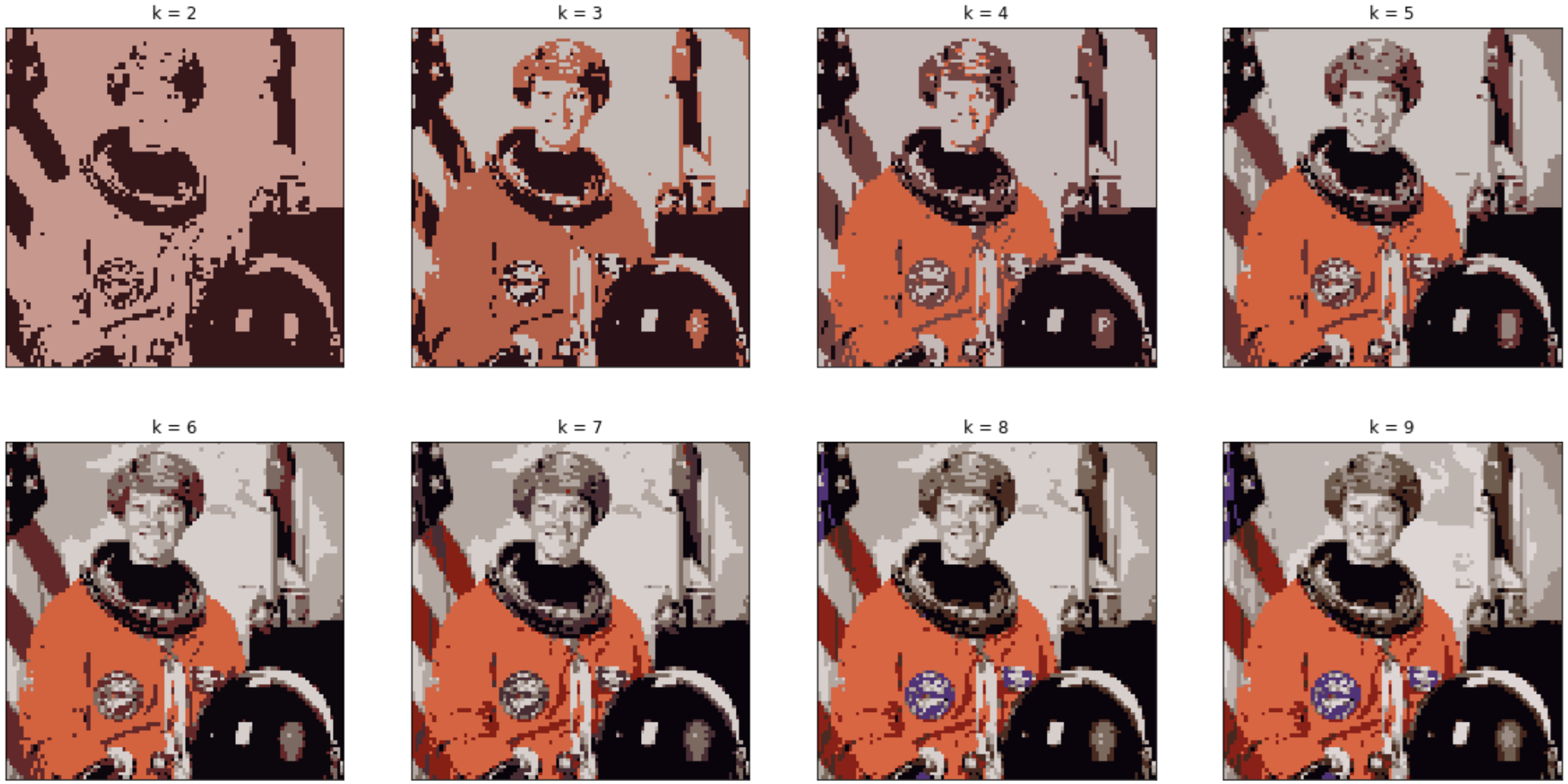
- Pick k cluster centers (random, or some good heuristic)
- For each pixel color value - assign the closest cluster
- Recompute cluster center (mean) from assigned pixels.
- Repeat until converged.







Segmentation as clustering / K-Means

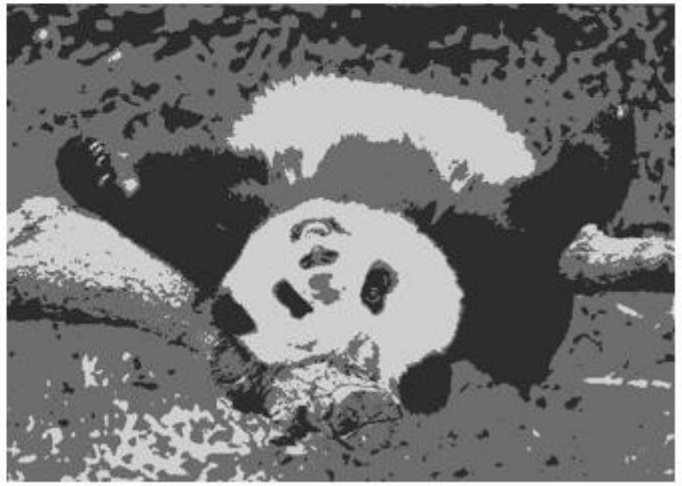


Segmentation as clustering / K-Means

How to choose the number of clusters?

k=2

k=3



Try different numbers of clusters and look at performance more clusters typically work better

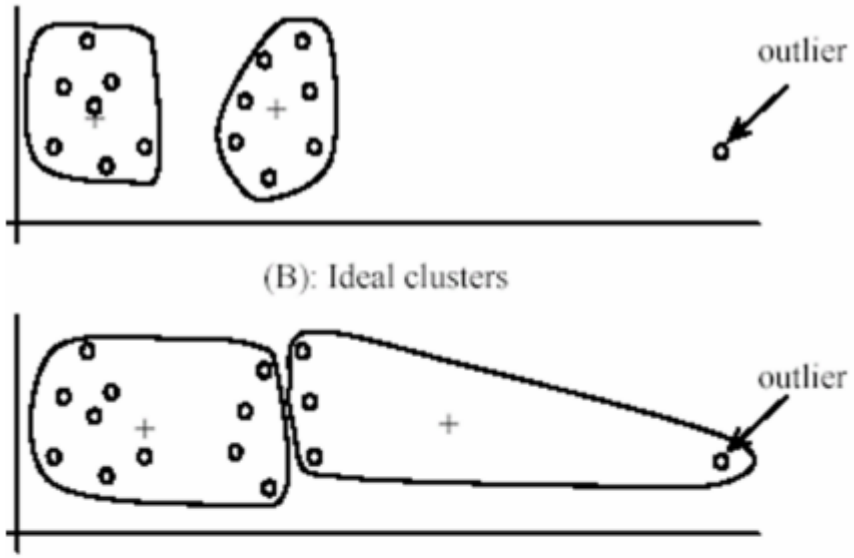
Segmentation as clustering / K-Means

Pros

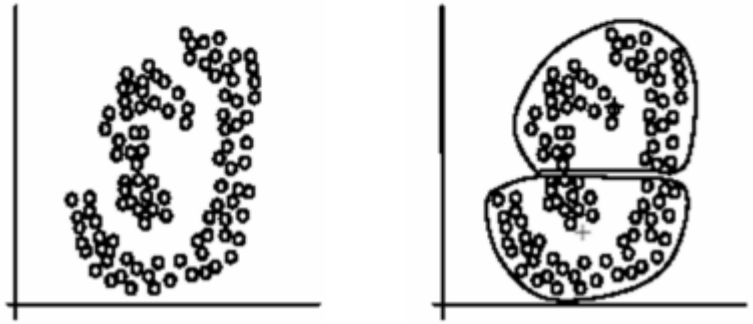
- Finds cluster centers that minimize conditional variance (good representation of data)
- Simple and fast, Easy to implement

Cons

- Need to choose K
- Sensitive to outliers
- All clusters have the same parameters (e.g., distance measure is non-adaptive)
- Can be slow: each iteration is O(KNd) for N dimensionl points



(B): Ideal clusters



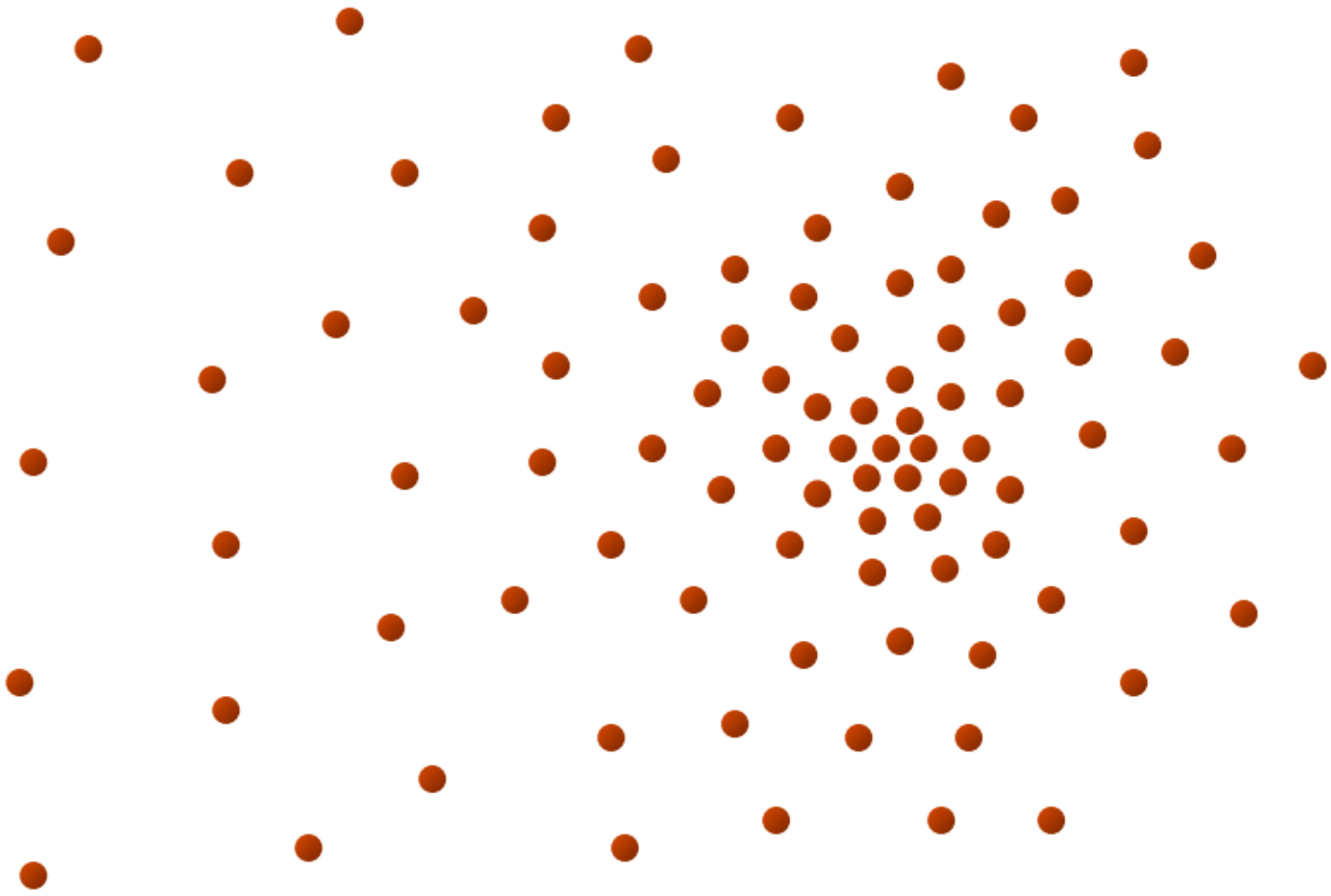
(A): Two natural clusters (B): k-means clusters

Segmentation as clustering / MeanShift

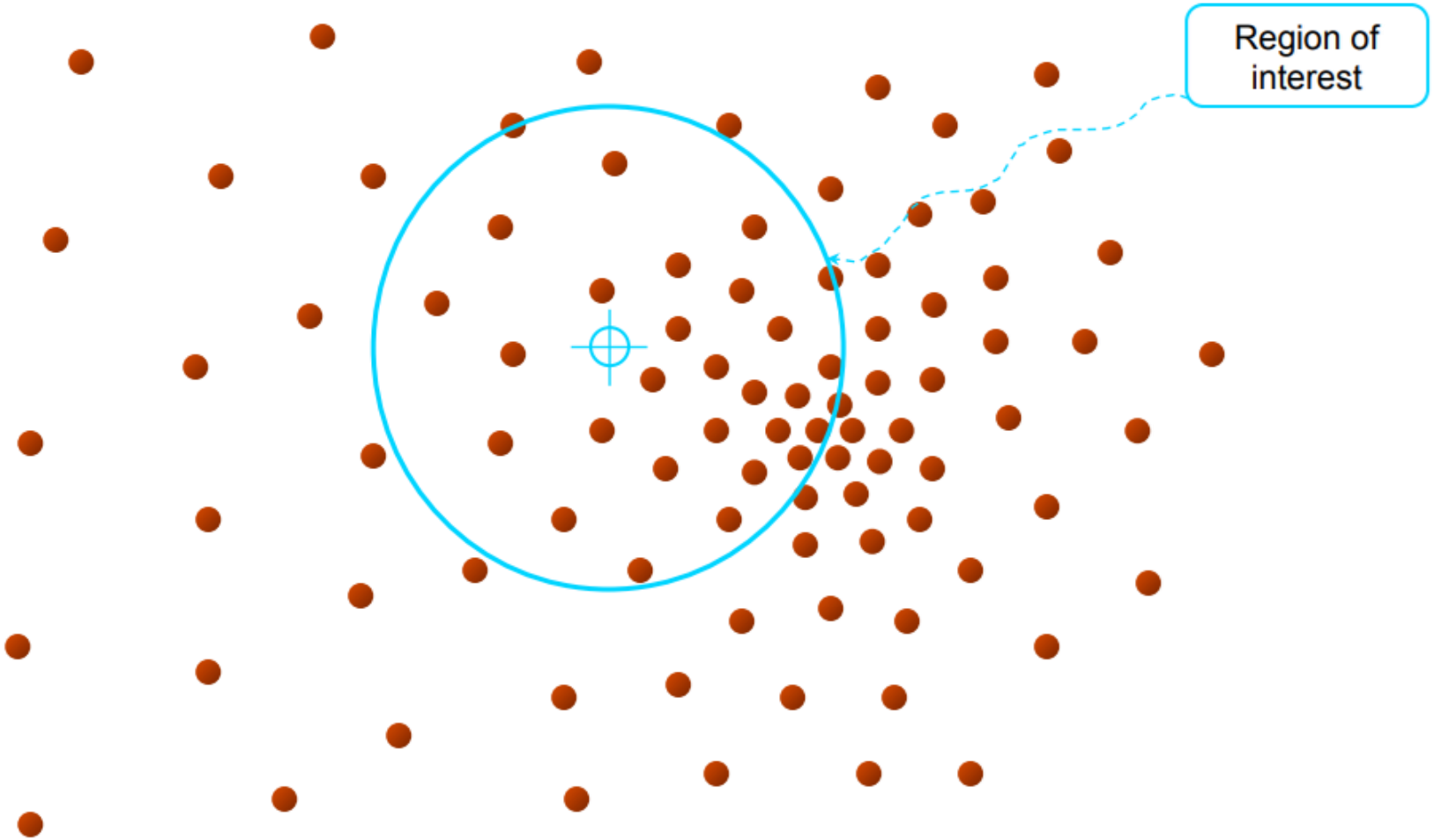
Iterative Mode Search

1. Initialize random seed, and window W
2. Calculate center of gravity (the "mean") of W: $\sum_{x \in W} xH(x)$
3. Shift the search window to the mean
4. Repeat Step 2 until convergence

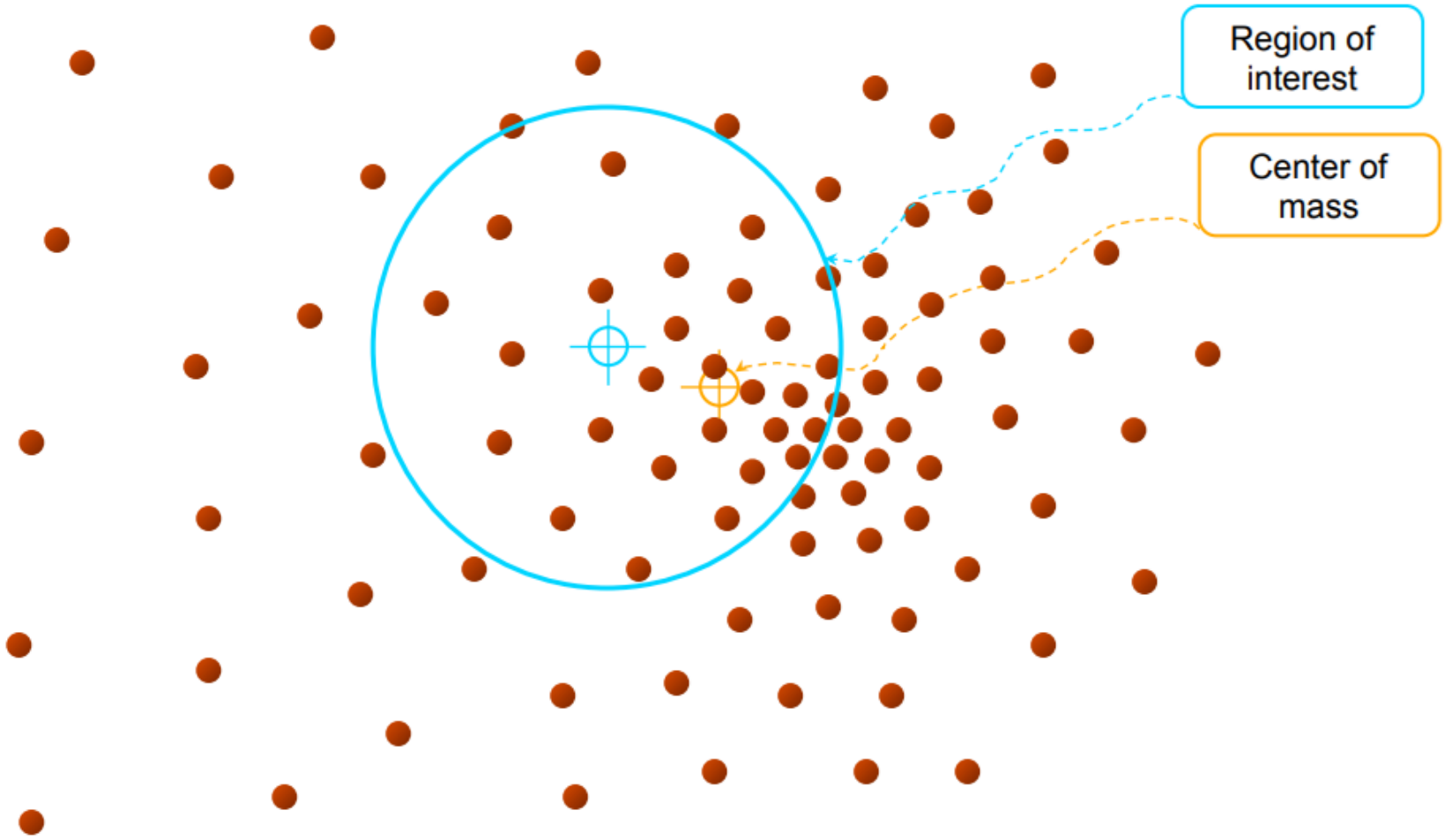
Segmentation as clustering / MeanShift



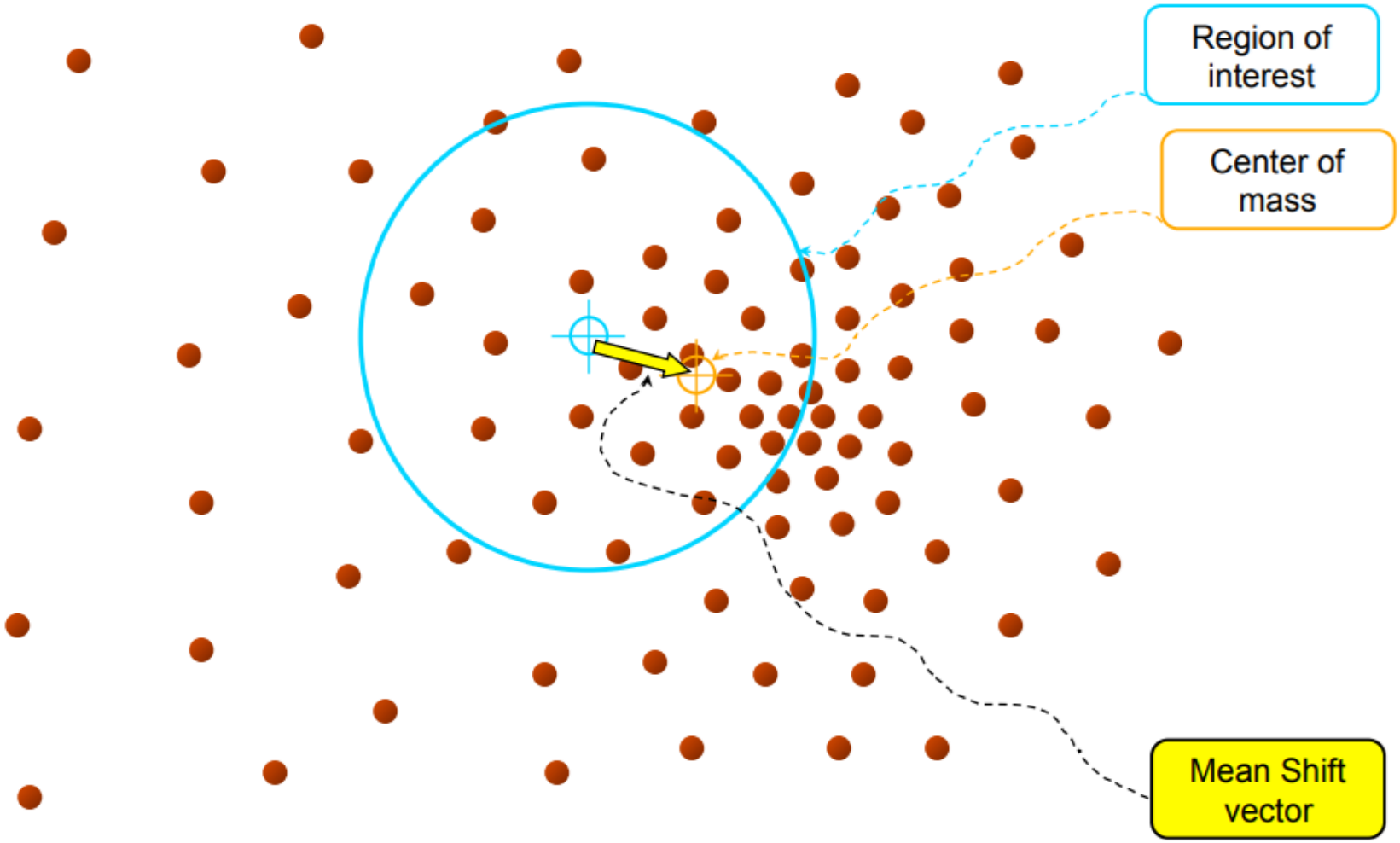
Segmentation as clustering / MeanShift



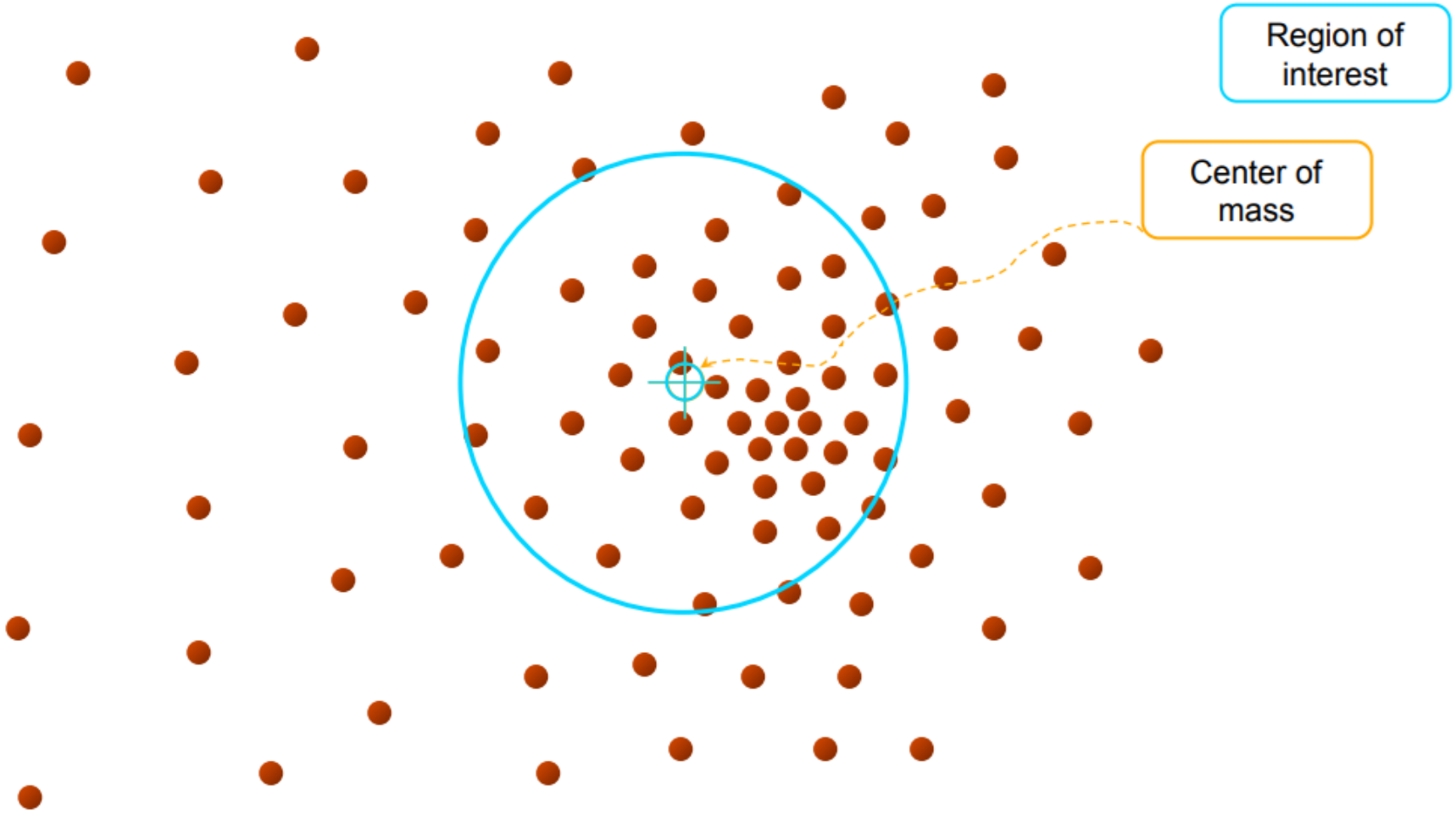
Segmentation as clustering / MeanShift



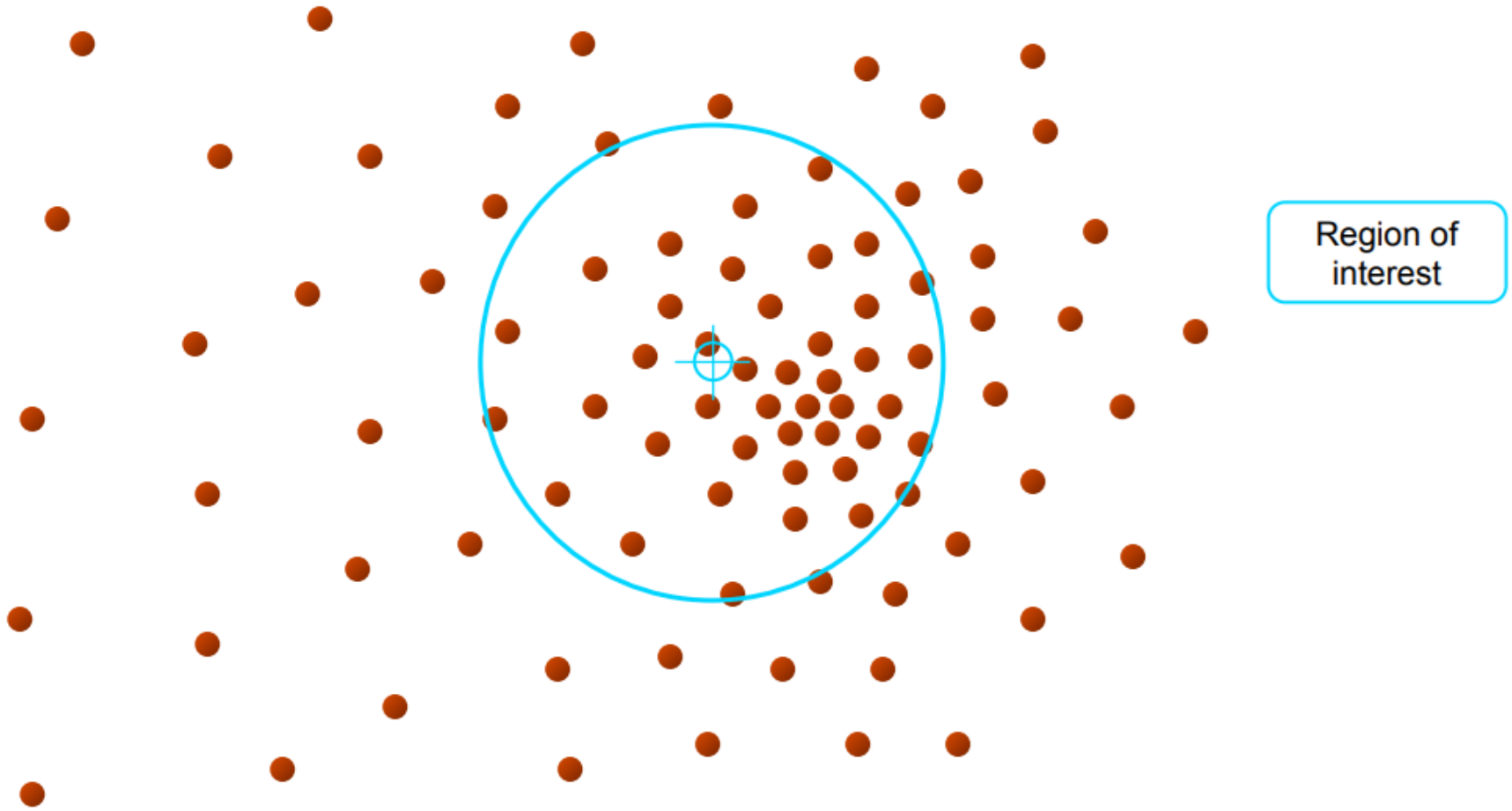
Segmentation as clustering / MeanShift



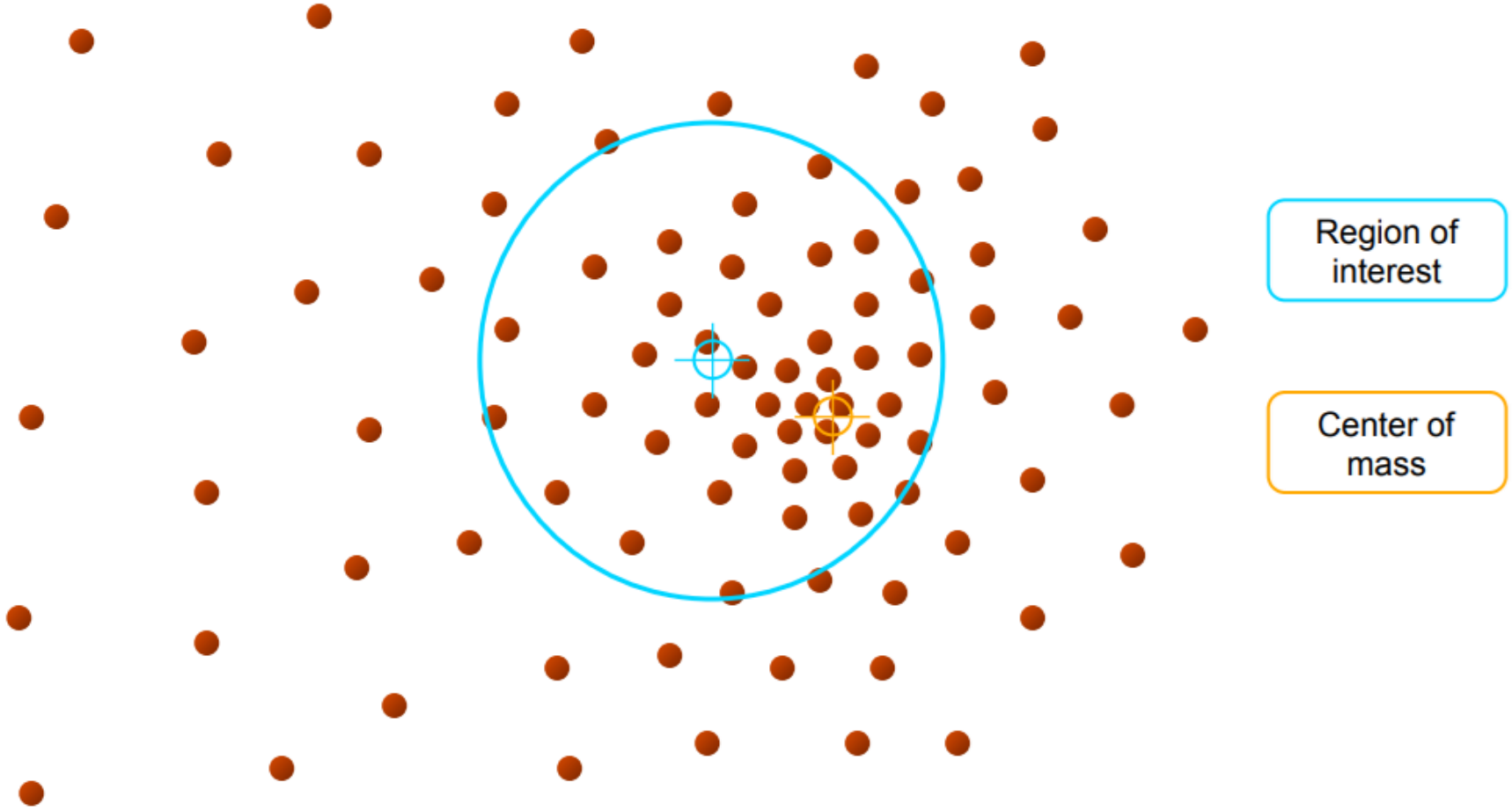
Segmentation as clustering / MeanShift



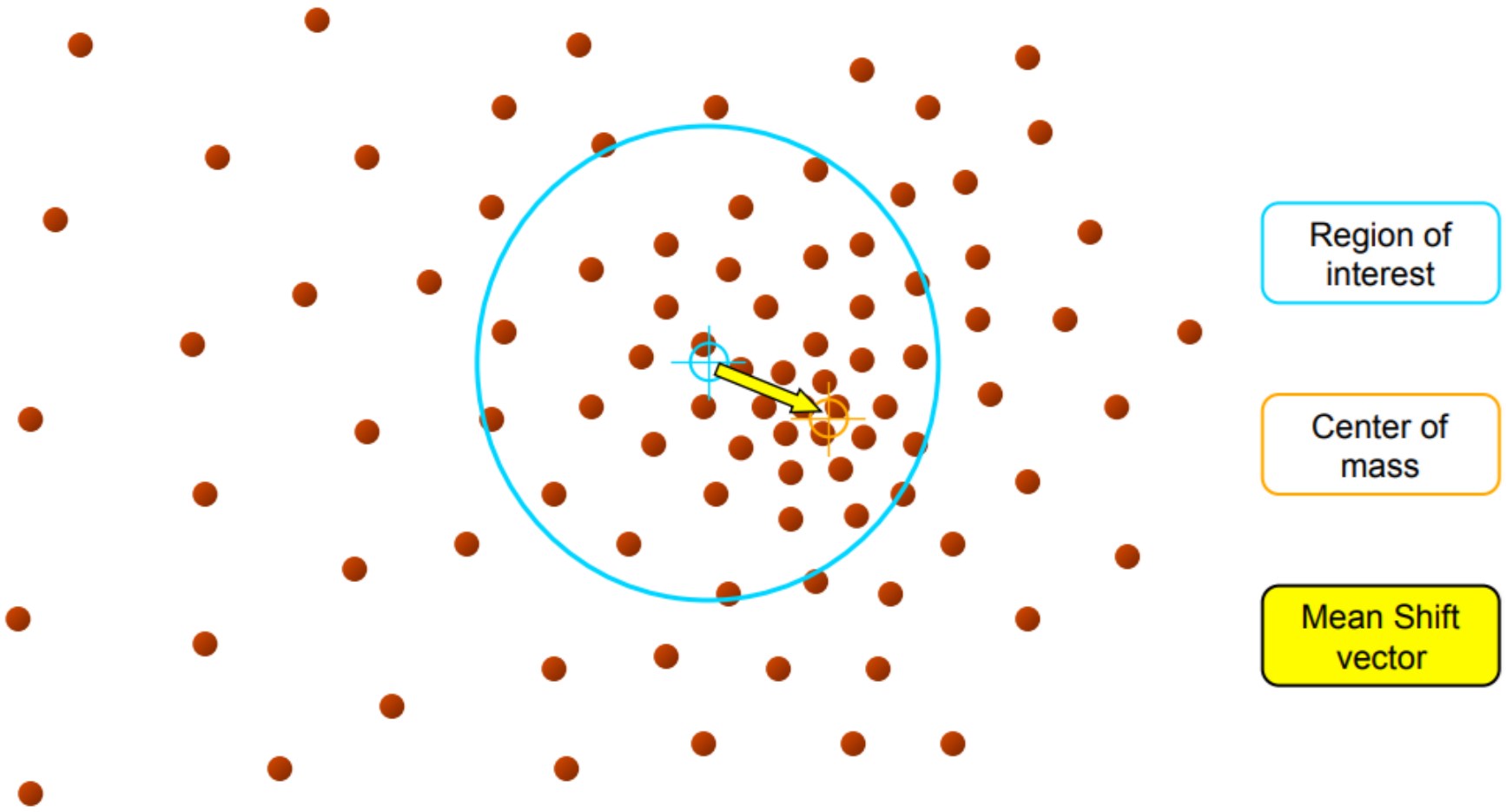
Segmentation as clustering / MeanShift



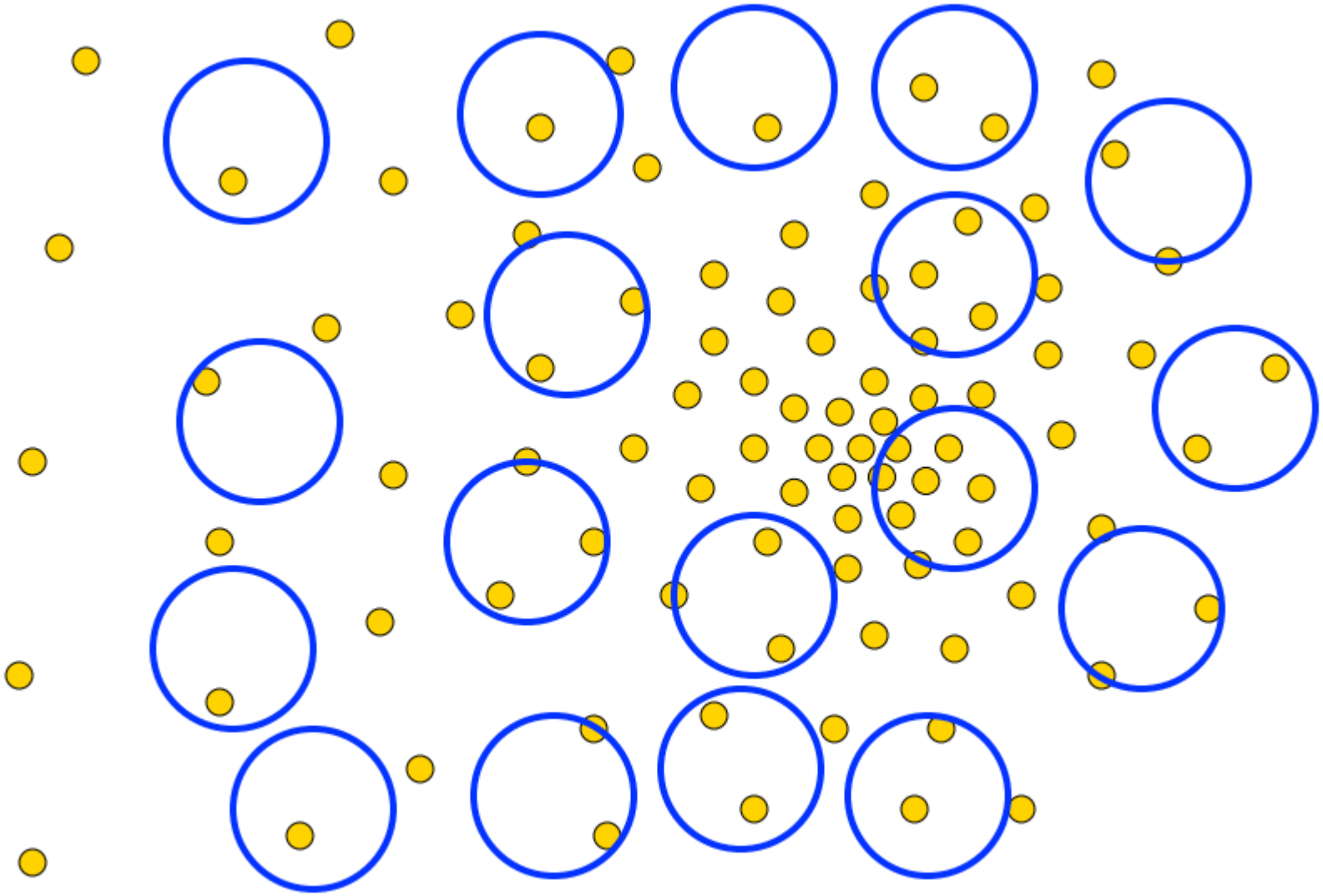
Segmentation as clustering / MeanShift



Segmentation as clustering / MeanShift



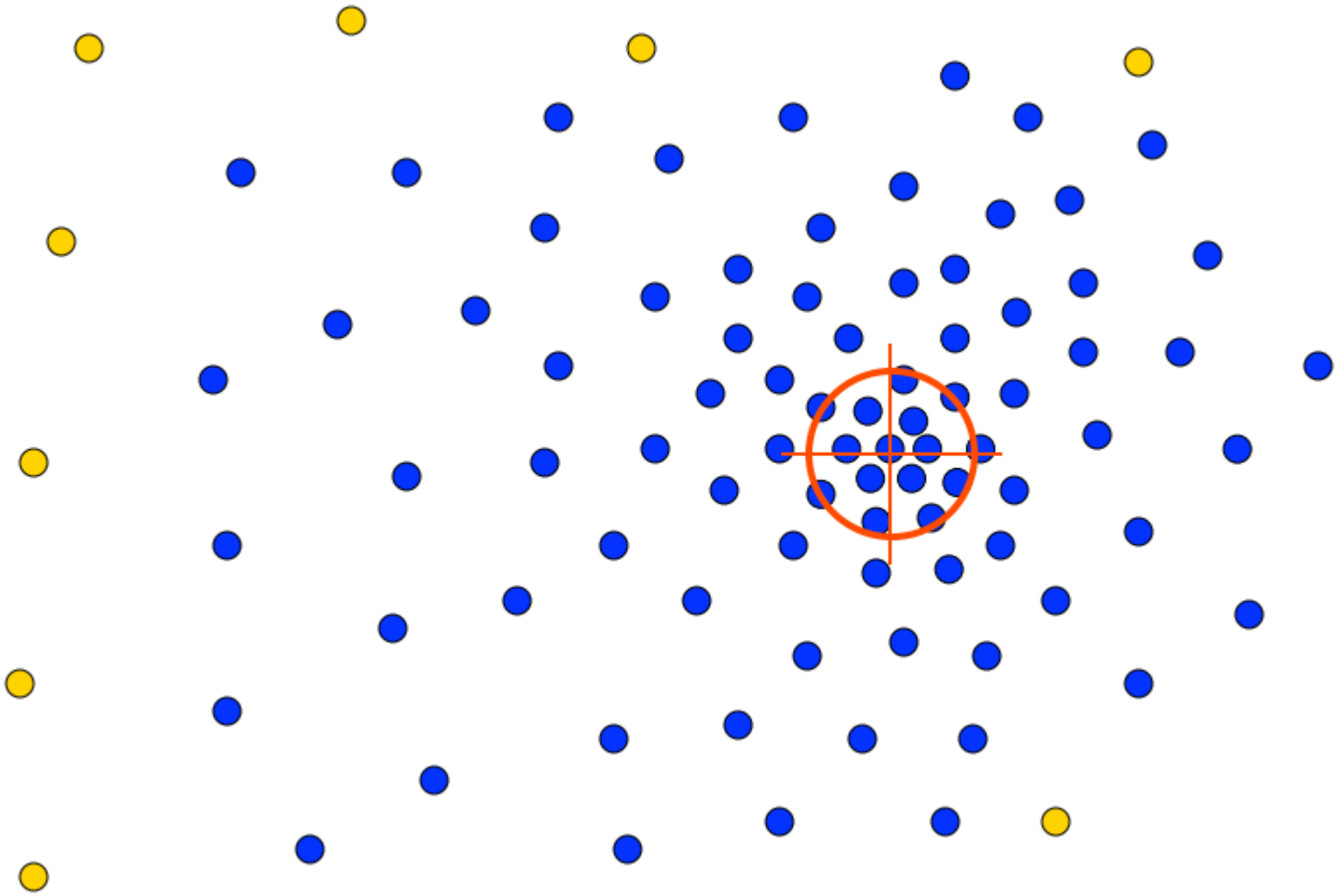
Segmentation as clustering / MeanShift



Tessellate the space with windows

Run the procedure in parallel

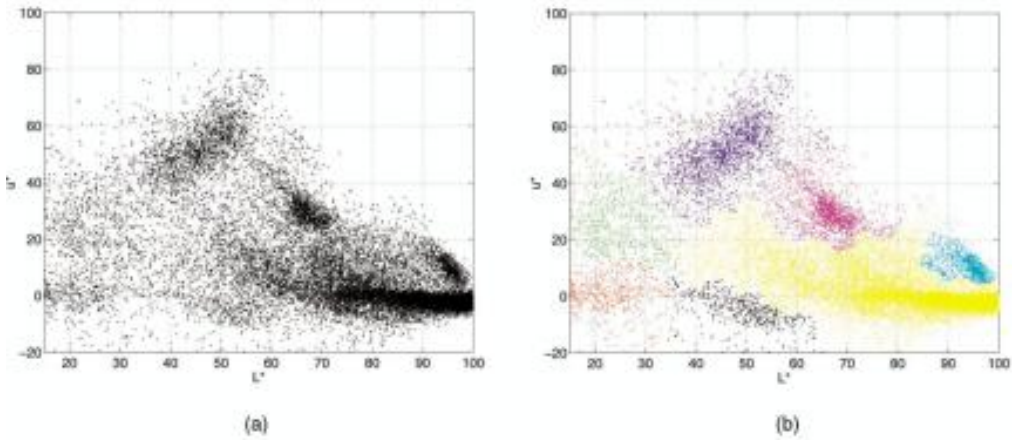
Segmentation as clustering / MeanShift

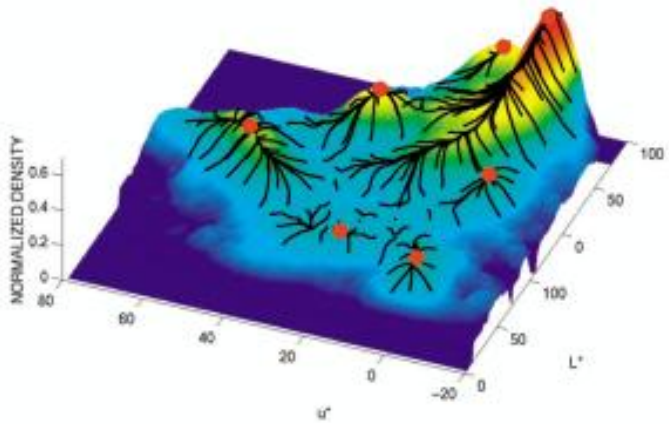


The blue data points were traversed by the windows towards the mode.

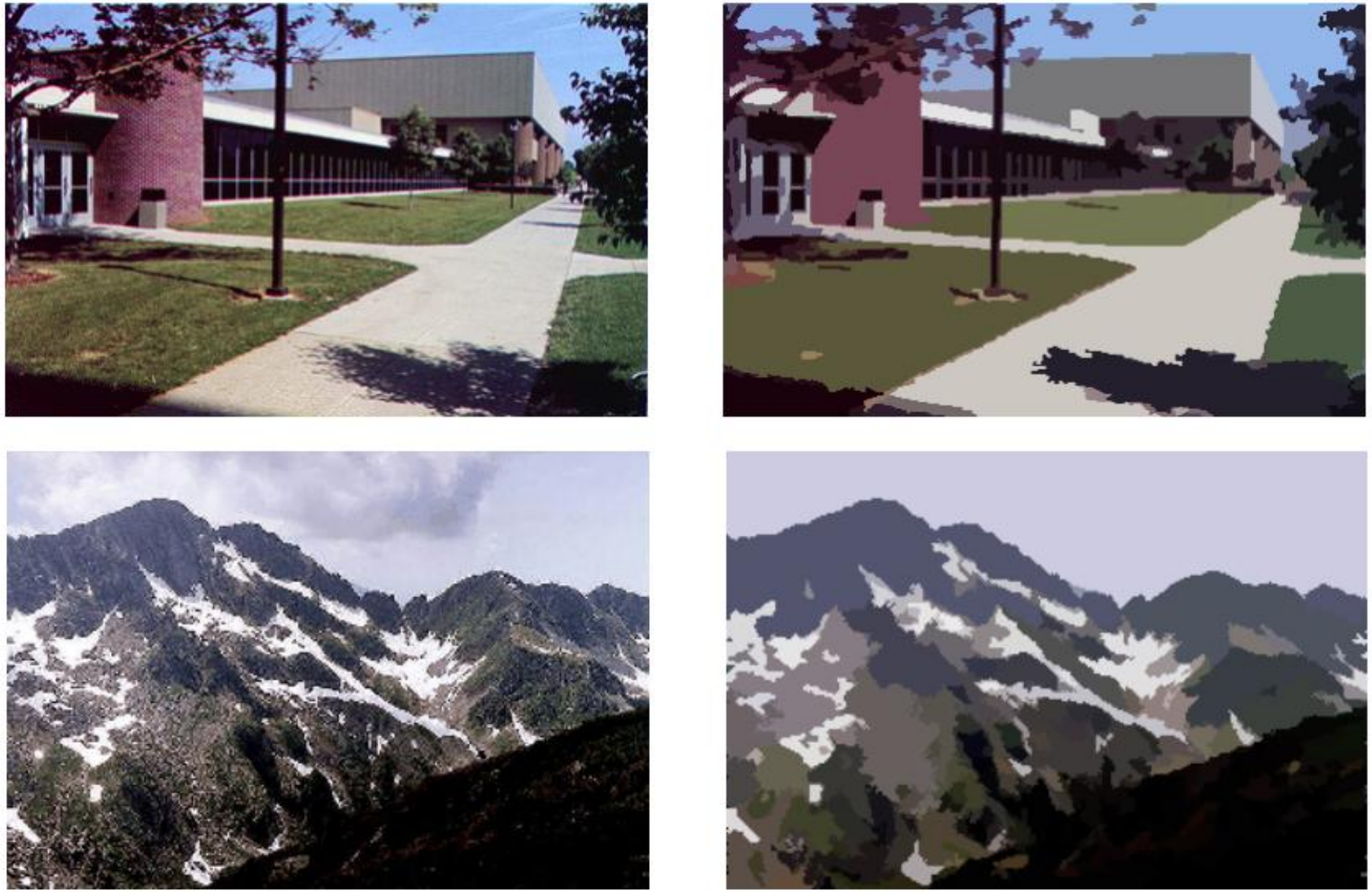
Segmentation as clustering / MeanShift

1. Find features (color, gradients, texture, etc)
2. Initialize windows at individual pixel locations
3. Perform mean shift for each window until convergence
4. Merge windows that end up near the same "peak" or mode

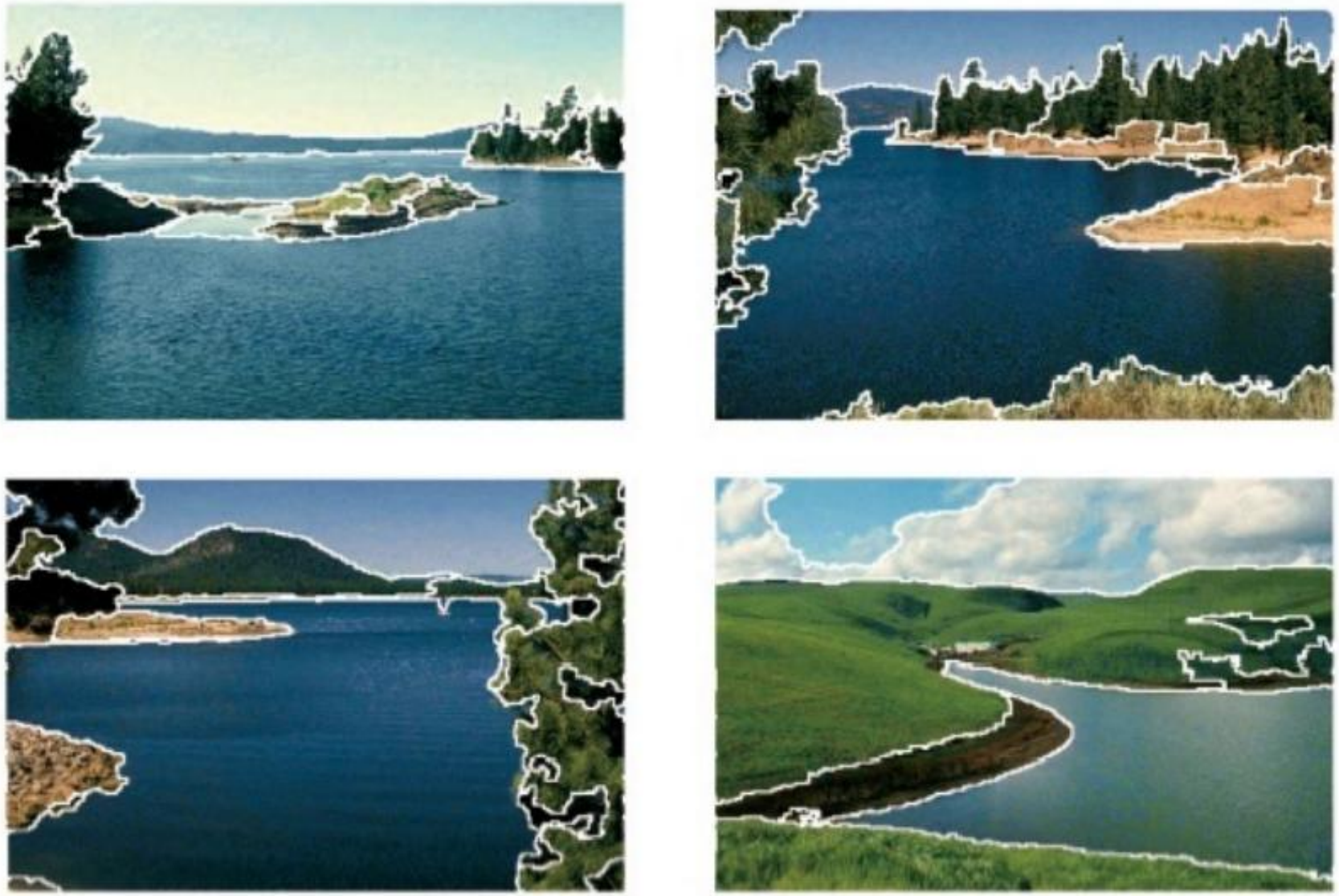




Segmentation as clustering / MeanShift



Segmentation as clustering / MeanShift



Segmentation as clustering / MeanShift

Pros

- General, application - independent tool
- Model-free, does not assume any prior shape (spherical, elliptical, etc.) on data clusters
- Just a single parameter (window size w) : w has a physical meaning (unlike k means)
- Finds variable number of modes
- Robust to outliers

Cons

- Output depends on window size
- Window size (bandwidth) selection is not trivial
- Computationally (relatively) expensive (~2s/image)
- Does not scale well with dimension of feature space

Segmentation as clustering

Pro:

- Generally simple
- Can handle most data distributions with sufficient effort.

Con:

- Hard to capture global structure
- Performance is limited by simplicity